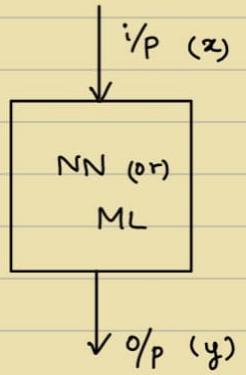
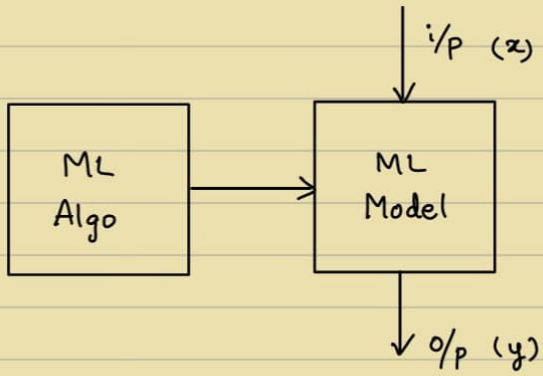
5/1

SMAI



Traditional Programming } → Output = ?
Given input & Program

- 

Ex. Sorting, Factorial

Easy for a human to understand and do

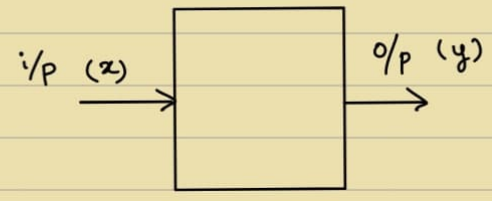
- E.g. NLP → Human cannot understand & do in the brain
  i.e. Voice → Wave transform → Numbers (data : x_i)

  Face Recognition → 100s of attributes
  i.e. Face → Pixels → Numbers (data : x_i)

  Data : $\{(x_i, y_i)\}$

φ) What makes a chair a chair ?
(i) 4 legs ? But cow has
(ii) Flat surface ? But Desk has
⋮
Exception everywhere , Only brain can know
∴ ML Model

AI : Blackbox



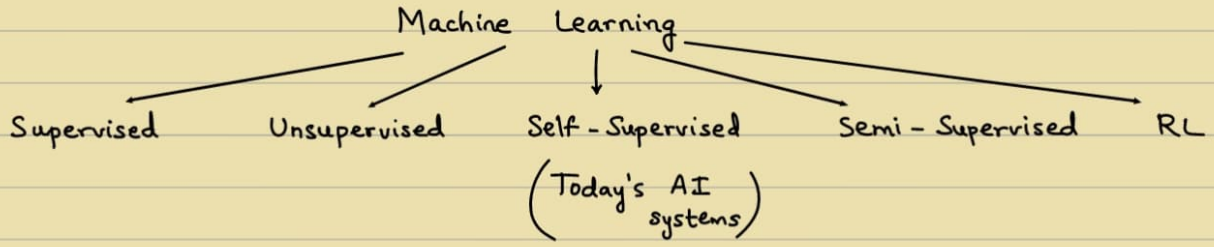
∴ ML Solution :

$$y = f(w, x)$$

↳ Learnable parameters

Training : Requires lot of data , GPU & Time
Inference : Testing

→



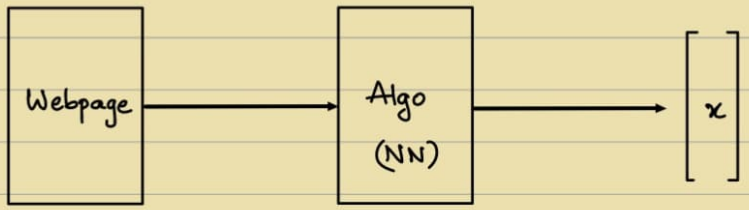
→ Represent data using Vector
↳ Embedding

$$\begin{bmatrix} x \end{bmatrix}$$

e.g. Webpage ⟶ Embed into a fixed dimensional vector which has the essence of the webpage

e.g. Sentiment Analysis , Speech recognition

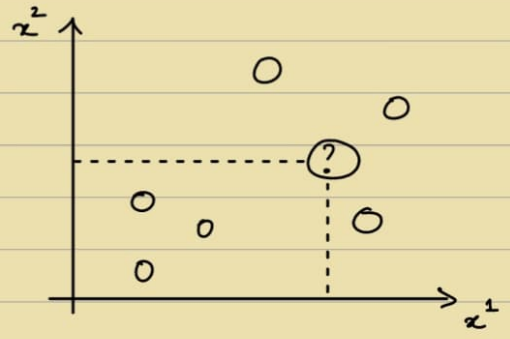
↳ Represent tweet as a 'd' dimensional vector.
$x \in R^d$



'y' could be scalar / string / Tree / Graph
x is generally vectors / Sequence of vectors

A4 Sheet ⟶ Define problem formally

→ Nearest Neighbour Classification :



Compute distance from all samples
& then sort
& find nearest

Disadvantage : Memory usage ↑
More computation

∴ Not efficient
∵ No training

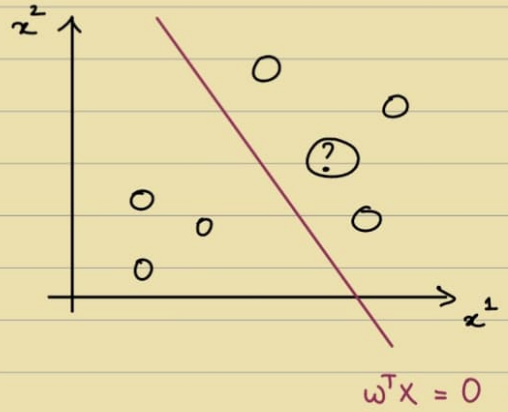
∴ Lazy algorithm

$$x = \begin{bmatrix} x^1 \\ x^2 \end{bmatrix}$$

Extension : 3 - NN , 5 - NN

∴ k - Nearest neighbours - Use Eucledian distance

→ sign $(w^T x)$:

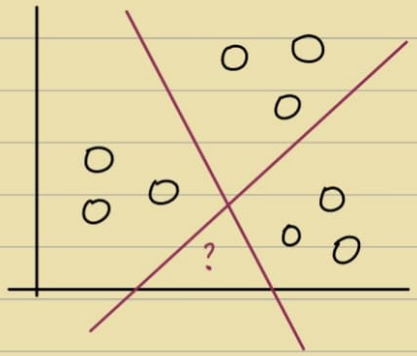


$$\begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix}^T \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = 0$$

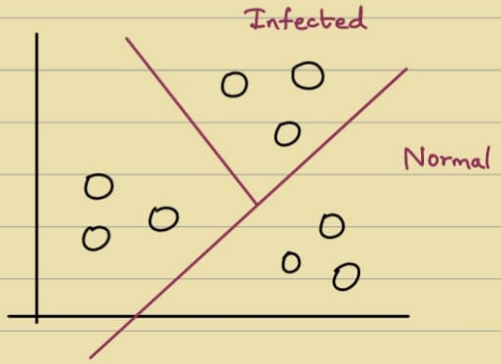
'd' multiplications
'd-1' additions

$$\Rightarrow w_1 x_1 + w_2 x_2 + w_3 x_3 = 0$$

For Multi - class classification , say 3
KNN works.
sign$(w^T x)$ can be extended

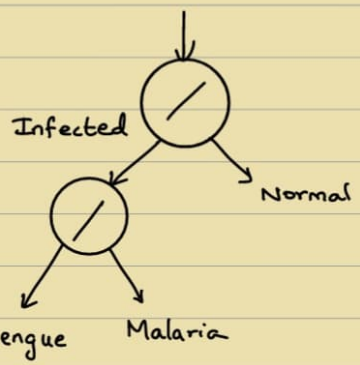


One solution :

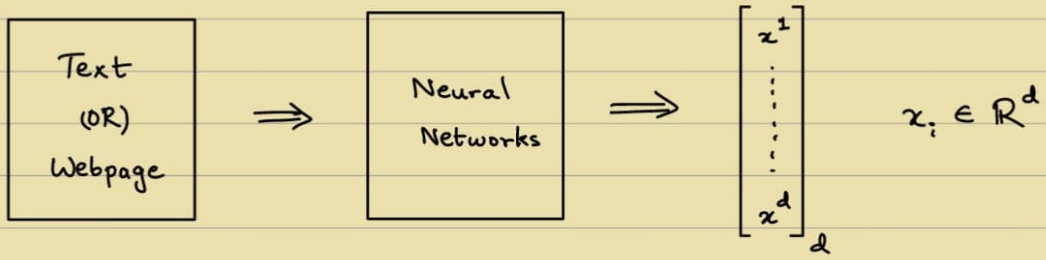


1 Solution :

Tree - like



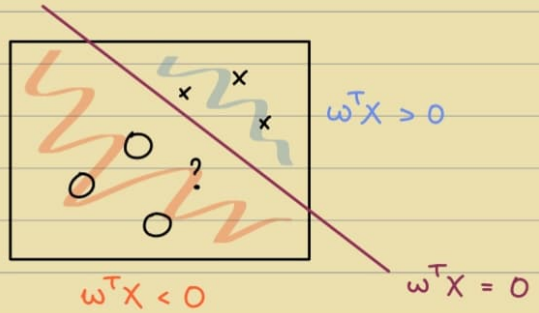
8/1



Similarity

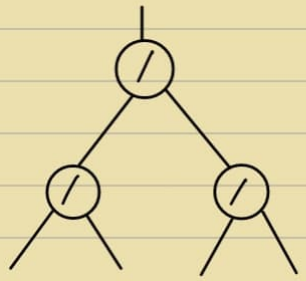
↳ Distance (Triangular Inequality)
[Eucledian - Geometric Intuition]

- Cosine similarity is also another method $(x^T y)$



→ Multi - class classifiers :

(1) Heirarchial



e.g. k classes , $\log(k)$ classifiers

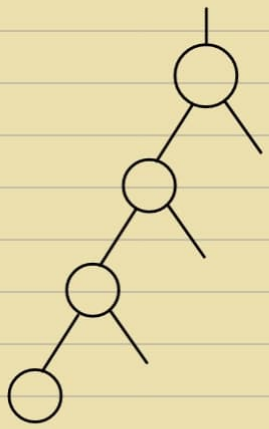
Cons : Many calculations , Not efficient
Pros : Lesser Inference time

(2) Majority Voting :

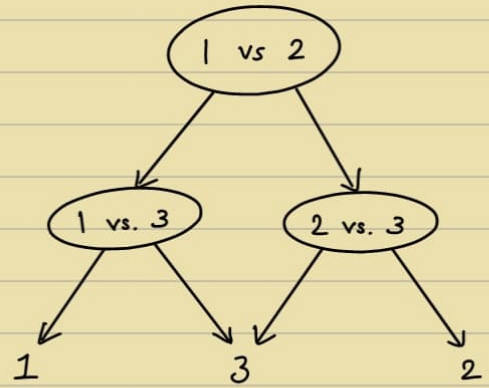
${}^{k}C_{2}$ classifiers , k : no. of classes

Computational complexity : k^2
Accuracy - wise

(3) 1 vs. Rest

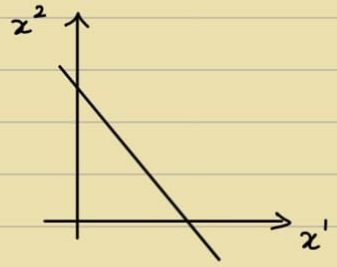


- Optimization : DAG (Directed Acyclic Graph)



- $w^T X = 0$

  $w_1 x^1 + w_2 x^2 + w_3 = 0$



$$\begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ 1 \end{bmatrix}$$

- x vector is being improved as it passes through each layer

$$\begin{bmatrix} x \end{bmatrix} \xRightarrow{\phi} \begin{bmatrix} x' \end{bmatrix}$$

$X' = WX$

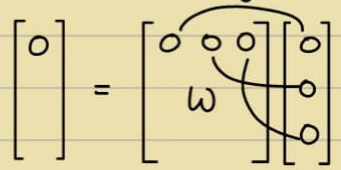
↳ Linear Transformation

X_d , W_{mxd}

$m \ll d$: Dimensional Reduction (e.g. PCA)

$m > d$: New enriched features (Weighted combination)

e.g. FC , MLP



Numerically useful :

$\mu = 0$ & $\sigma^2 = 1$ [Standard Scalar]

[OR]

Scale to range $[0, 1]$

$\| L_2 \| = 1$

Normalization

e.g. Reducing :

$\{10, 25, -5, 7.5, 10\}$

$x_1 \quad x_2 \quad x_3 \quad x_4 \quad x_5$

$$x' = \frac{x - \min}{\max - \min} \Longrightarrow [0, 1]$$

$$x' = \frac{x - \mu}{\sigma} \Longrightarrow \mu = 0 \ \& \ \sigma^2 = 1$$

Stable Algorithms ∵ Numbers are well-behaved & ∴ Better result

- Normalizing vectors :

$$x = \begin{bmatrix} 2 \\ 1 \\ 1.5 \end{bmatrix}$$

One method $\longrightarrow \frac{x}{\|x\|}$

$X_1, X_2, \ldots, X_5$

$\downarrow$

$\overline{X_1}, \overline{X_2}, \ldots, \overline{X_5}$

$\downarrow$

$\overline{X_1} - \mu, \overline{X_2} - \mu, \ldots \overline{X_5} - \mu$

$$\mu = \frac{X_1 + X_2 + \ldots + X_5}{5}$$

For variance $\Longrightarrow$ Divide by σ^2

But covariance can exist.

$$\Sigma^{-1}[\overline{X_1} - \mu]$$

Covariance Matrix

$$\mu = \frac{1}{N}\sum_{i=1}^{N} X_i$$

$$\Sigma = \frac{1}{N}\sum_{i=1}^{n}[X_i - \mu][X_i - \mu]^T$$

$$\Sigma^{-1}_{d \times d}, \; X_{i_{d \times 1}}, \; \mu_{d \times 1}$$

$\longrightarrow$ Evaluation Metrics :

- Accuracy
- Confusion Matrix
- Precision
- Recall
- AP
- F - score

Classification: Accuracy, Confusion Matrix, Precision

Ranking / Retrieval: Precision, Recall, AP, F - score

$$\text{Accuracy} : \frac{\text{Total correct}}{\text{Total}}$$

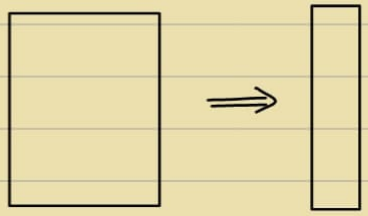
$\longrightarrow$ Reduction in KNN :

Euclidian Distance b/w X and Y $= [X_1 - X_2]^T[X_1 - X_2]$

$$[X_1 - X_2]^T_{1 \times d} \; D_{d \times d} \; [X_1 - X_2]_{d \times 1}$$

Diagonal Matrix, $D = \Sigma^{-1}$

12/1



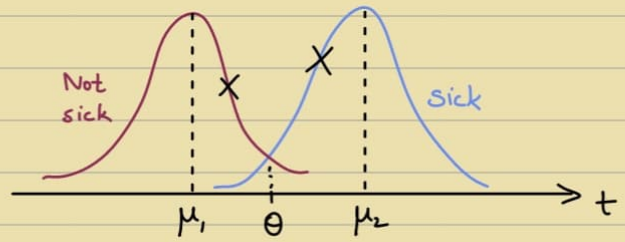
$D = \{x_i, y_i\}$

$i = 1, 2, \ldots, N$

$x \in \mathbb{R}^d$

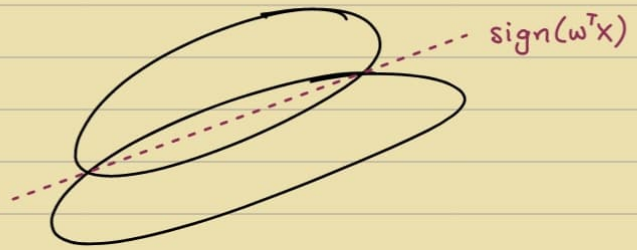
$y \in \mathbb{R}$

$\text{sign}(\omega^T x) \longleftarrow f(\omega, x)$



$P(\text{sick} \mid x) \geq P(\text{Not sick} \mid x)$

$\therefore$ Sick , else not sick

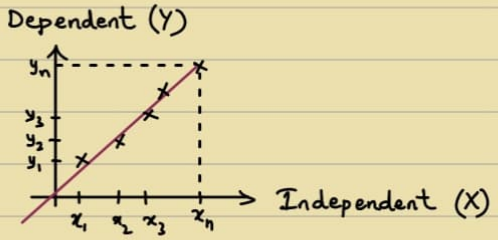


- Spec. of an ML Software : Accuracy & Precision
  [98% accurate 95% of the time]

  PAC : Probablitically Approximate.

→ Regression :

$y \in \mathbb{R}$



Do well on Unseen data } ⇒ Generalisation

$\hat{y}$: Predicted

MSE Loss : $(y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2 + \ldots + (y_{n-1} - \hat{y}_{n-1})^2$

minimise MSE loss by warying ω ⇒ $\min_{\omega} L(y, \hat{y})$

$\hat{y} \leftarrow \omega^T x$

$f(\omega, x)$

$\omega^* = \arg\min_{\omega} L(y, \hat{y})$

$$\Rightarrow \omega^* = \arg\min_{\omega} \frac{1}{N} \cdot \sum_{i=1}^{N} (y_i - \omega^T x_i)^2$$

$$y = \begin{bmatrix} \omega_1 \\ \omega_2 \end{bmatrix} \begin{bmatrix} x_1 \\ 1 \end{bmatrix}$$

[Discrete optimization]

$$y = \omega_1 x_1 + \omega_2$$

Loss function should be differentiable.

$$\therefore \omega^* = \frac{1}{N} \cdot \arg\min_{\omega} \left(\sum_{i=1}^{N} (y_i - \omega^T x_i)^2 \right)$$

Accuracy cannot be taken as loss function ∵ Non-differentiable

→ Closed Form Solution :

$$\hat{Y}_1 = \omega^1 x_1^1 + \omega^2 x_1^2 + \omega^3 x_1^3 + \omega^4 1$$
$$\hat{Y}_2 = \omega^1 x_2^1 + \omega^2 x_2^2 + \omega^3 x_2^3 + \omega^4 1$$
$$\hat{Y}_3 = \omega^1 x_3^1 + \omega^2 x_3^2 + \omega^3 x_3^3 + \omega^4$$

$$\underset{n\times 1}{\begin{bmatrix} Y \end{bmatrix}} = \underset{n\times d}{\begin{bmatrix} X \end{bmatrix}} \underset{d\times 1}{\begin{bmatrix} \omega \end{bmatrix}}$$

$$\frac{1}{N} \left[Y - \hat{Y} \right]^T \left[Y - \hat{Y} \right]$$

$$\Rightarrow \left[Y - X\omega \right]^T \left[Y - X\omega \right]$$
$$\Rightarrow Y^T Y + (X\omega)^T (X\omega) - 2(X\omega)^T Y$$
$$\Rightarrow Y^T Y + \omega^T X^T X \omega - 2\omega^T X^T Y$$

$$\frac{\partial}{\partial \omega} \left(Y^T Y + \omega^T X^T X \omega - 2\omega^T X^T Y \right)$$

$$= \frac{\partial}{\partial \omega} (\omega^T X^T X \omega) - 2X^T Y$$

$$= 2X^T X \omega - 2X^T Y \overset{\text{set}}{=\!=} 0$$

$$X^T X \omega = X^T Y$$
$$\Rightarrow \omega = \underbrace{(X^T X)^{-1}}_{d\times d} \underbrace{X^T}_{d\times n} \underbrace{Y}_{n\times 1}$$

$\omega^T A \omega$ → Quadratic form in matrices

$$\frac{\partial}{\partial \omega} (\omega^T A \omega) = 2A\omega$$

$\left[\omega : d\times 1,\ A : d\times d \right]$

$\{ A^T B = B^T A \}$

→ Gradient descent :

Start with ω^0

Iterate (To minimize J)

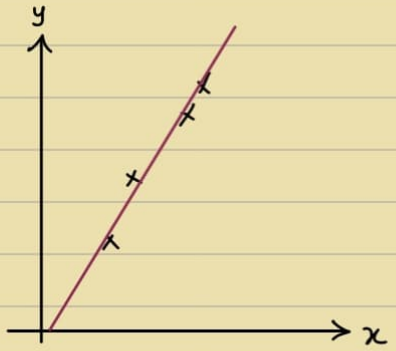
$$\omega^{k+1} \leftarrow \omega^k - (\nabla J)(lr)$$

(lr : Learning Rate)

$$\min_{\omega} J = \min_{\omega} \frac{1}{N} \sum_{i=1}^{N} (y_i - \omega^T x_i)^2$$

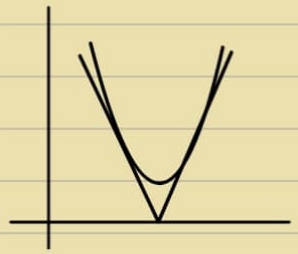
$$\nabla J = \frac{2}{N} \sum_{i=1}^{N} (y_i - \omega^T x_i)(-x_i)$$

19/1



$$y = \omega' x + \omega^0$$

$$\frac{1}{N} \cdot \sum_{i=1}^{N} (\hat{y}_i - y_i)^2$$



$$y = \omega^T X = f(\omega, X)$$

$$= \begin{bmatrix} \omega_1 \\ \omega_2 \end{bmatrix} \begin{bmatrix} x \\ 1 \end{bmatrix}$$

$$\left.\begin{aligned} y_1 &= \omega' x_1 + \omega^0 \\ y_2 &= \omega' x_2 + \omega^0 \\ &\vdots \quad\quad \vdots \quad\quad \vdots \\ y_N &= \omega' x_N + \omega^0 \end{aligned}\right\} \Longrightarrow \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} = \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_N & 1 \end{bmatrix} \begin{bmatrix} \omega' \\ \omega^0 \end{bmatrix}$$

$$\Rightarrow \quad Y = X\omega$$

$$[Y - \hat{Y}]^T [Y - \hat{Y}]$$
$$[Y - X\omega]^T [Y - X\omega]$$

- Closed Form Sol. :

$$\omega^* = (X^T X)^{-1} X^T Y$$

- Gradient Descent Sol. :

(i) Initialize ω^0

(ii) $\omega^{k+1} \longleftarrow \omega^k - \eta \nabla J$

$$\min_{\omega} J(\omega) = \min_{\omega} \frac{1}{N} \sum_{i=1}^{N} (y_i - \omega^T x_i)^2$$

$$\nabla J = \frac{2}{N} \sum_{i=1}^{N} (y_i - \omega^T x_i)(-x_i)$$

→ Regularisation :

Some features are not as useful as the others.

$$J = f(\omega) + \lambda g(x)$$

$$J = f(\omega) + \lambda \|\omega\|$$

Regularize with L2, L2 (ideally L0) norms

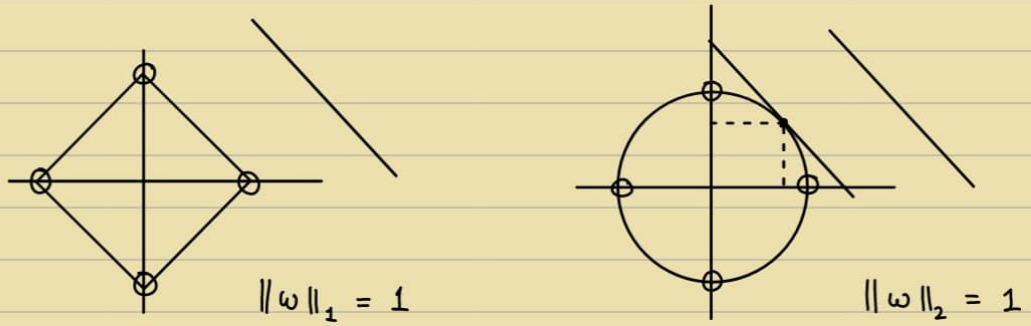
$$\|x\| = \left(\sum_{i=1}^{d} |x_i|^p\right)^{1/p}$$

L1 ($d=2$): $\|x\|_1 = |x_1| + |x_2|$

L2 ($d=2$): $\|x\|_2 = \sqrt{x_1^2 + x_2^2}$

- L0 norm : Counts the non-zero elements in the vector
- L_∞ norm : Max. Abs. value of the vector ← max. norm

→ Pseudo-norms



→ Better in eliminating Sparsity

$$\min J(\omega) \quad \text{S.T.} \quad \|\omega\| = 1$$

- **Ridge Regression** :

  Linear Regression normalized with L_2 Norm

$$J = \sum_{i=1}^{N} (y_i - \omega^T x_i)^2 + \lambda \cdot \sum_{j=1}^{d} \omega_j^2$$

$$= \sum_{i=1}^{N} (y_i - \omega^T x_i)^2 + \lambda \cdot \|\omega\|_2^2$$

$$J(\omega) = \frac{1}{N} \left(Y^T Y + \omega^T X^T X \omega - 2\omega^T (X^T Y) + \lambda \omega^T \omega\right)$$

Optimizing $\Rightarrow$ $\omega = (X^T X + \lambda I)^{-1} X^T Y$

- **Lasso Regression** :

  Linear Regression normalized with L_1 Norm

$$J = \sum_{i=1}^{N} (y_i - w^T x_i)^2 + \lambda \cdot \sum_{j=1}^{d} |w_j|$$

$$= \sum_{i=1}^{N} (y_i - w^T x_i)^2 + \lambda \cdot \|w\|_1^2$$

  Solution to Lasso — Outside the scope of course

  Optimization leads to zero of irrelevant features.

→ **Eigenvalues & Eigenvectors** :

$$Av = \lambda v$$

v → Eigenvector (v_i) ← Normalized to unit norm

λ → Eigenvalues (λ_i) ← Sorted in decreasing order

Max. $w^T A w$ S.T. $w^T w = 1$ $\left(\|w\|_1 = 1\right)$

Using Lagrangian (λ) :

$$J(w, \lambda) = w^T A w - \lambda (w^T w - 1)$$

Differentiate w.r.t w :

$$Aw = \lambda w$$

w → eigenvalue corresponding to the largest eigenvalue.

$\because$ max. $w^T A w$ S.T. $w^T w = 1$

$\Rightarrow$ max. $w^T \lambda w$

$\Rightarrow$ max. $w^T w \cdot \lambda$

$\Rightarrow$ max. λ ← $\therefore$ Pick largest λ

$Ax = 0$: Homogenous

↳ $m \times n$

Min. $\|Ax\|$ S.T $\|x\| = 1$

$$\Rightarrow \min_x \; x^T A^T A x - \lambda (x^T x - 1)$$

$$\Rightarrow 2AA^T x = 2\lambda x$$

$$\Rightarrow AA^T x = \lambda x$$

Assuming $y_i = mx_i + c$ form, derive update rule for m & c

Iterative : Better

Iterative & Closed Form ⟶ Same answer

↳ More Memory

Inverse is computationally expensive

But Parallelism

Processing in batches ⟶ Stochastic Gradient Descent

- e.g. $\alpha_i \in [0, 1]$

  0 : non-confident

  1 : confident

$$L = \frac{1}{N} \sum_{i=1}^{N} \alpha_i (y_i - \omega^T x_i)^2$$

- Closed Form

$$\underset{n\times 1}{\begin{bmatrix} Y \end{bmatrix}} = \underset{n\times n}{\begin{bmatrix} \alpha \end{bmatrix}} \underset{n\times d}{\begin{bmatrix} X \end{bmatrix}} \underset{d\times 1}{\begin{bmatrix} \omega \end{bmatrix}}$$

↳ Diagonal Matrix

- Gradient Descent :

$$\omega^{k+1} \leftarrow \omega^k - \nabla J$$

$$\min_{\omega} J = \min_{\omega} \frac{1}{N} \sum_{i=1}^{N} \alpha_i (y_i - \omega^T x_i)^2$$

$$\Rightarrow \nabla J = \frac{2}{N} \sum_{i=1}^{N} \alpha_i \cdot (y_i - \omega^T x_i)(-x_i)$$

$\begin{bmatrix} \\ \\ \\ \end{bmatrix} \in \mathbb{R}^d$

N

$X = \begin{bmatrix} & & \\ & & \\ & & \end{bmatrix}$

$N \times d$

$X^T X : d \times d$

$(N > d)$

$$\Sigma = \frac{1}{N} \sum_{i=1}^{N} [x_i - \mu][x_i - \mu]^T$$

$$\mu = \frac{1}{N} \sum_{i=1}^{N} x_i$$

$(d \times 1) \times (1 \times d) \equiv (d \times d)$ ← Rank = 1

→ One-hot representation:

$$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

- Rank of matrix after adding noise → no. of non-zero eigen values.

$$A : \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \xrightarrow[R_3 : R_3 - R_1]{R_2 : R_2 - R_1} \text{Rank} = 2$$

$A + \varepsilon B$, if $\varepsilon = 1$: Rank = 3

if $\varepsilon = 0.001$: Rank = 2

Rank of data matrix : $\min(N, d)$

- Numerical rank vs. Practical rank

Learning is easier when low rank structure, else classifier not very accurate

$Av = \lambda v$

↳ $N \times N$ [Positive Semidefinite]

$(x^T A x = 0)$

Eigen decomposition : $A = \sum_{i=1}^{d} \lambda_i v_i v_i^T$

$A = U \Lambda U^T$

Singular Value Decomposition (SVD)

$A = U \Sigma V^T$

5×3 or 3×5 $\quad 5 \times 3$ $\quad 3 \times 3$ $\quad 3 \times 3$

3×3 $\quad 3 \times 3$ $\quad 3 \times 3$

$A = UDV^T$

$\Rightarrow AA^T = (UDV^T)(VD^TU^T)$

$= UD^2U^T$

$A^TA = (VD^TU^T)(UDV^T)$

$= VD^2V^T$

$A_{N \times N}$ has closest low rank p, $p < N$, say A'

$A' = \arg \min_{M} \|A - M\|$

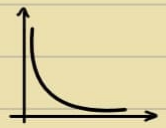
rank of M is p.

Solve using SVD :

$A \longrightarrow UDV^T$

$A' \leftarrow$ 

Sorted in descending order (eigen spectrum)



[Look at the data & Analyze it first !]

Tiny noise : Rank - 2 → 100

↙ Practical ↓ Numerical

$$A = X^T X$$

$$\Rightarrow A^T = (X^T X)^T = X^T X^{T^T} = X^T X = A$$

A : Symmetric

$$z^T (X^T X) z = (Xz)^T (Xz)$$

$\therefore X^T X$ is positively semi-definite.

$\therefore A$ is SPD.

$A_{d \times d}$

- Using SVD to find inverse

$$A = UDV^T$$

$$\Rightarrow A^{-1} = (V^T)^{-1} (D^{-1})(U^{-1})$$

$$= V(D^{-1}) U^T$$

- $\min_y \sum_{i=1}^{N} (x_i - y)^T (x_i - y)$

$$\Rightarrow \frac{\partial}{\partial y} \left(\sum_{i=1}^{N} x_i^T x_i + y^T y - 2 x_i^T y \right) \overset{\text{set}}{=\!=} 0$$

$$\Rightarrow 2y - 2x_i = 0$$

$$\therefore y = \frac{1}{N} \sum_{i=1}^{N} x_i$$

2/1

$X_1, X_2, \ldots\ldots, X_N \quad ; \quad X_i \in \mathbb{R}^d$

$\downarrow$

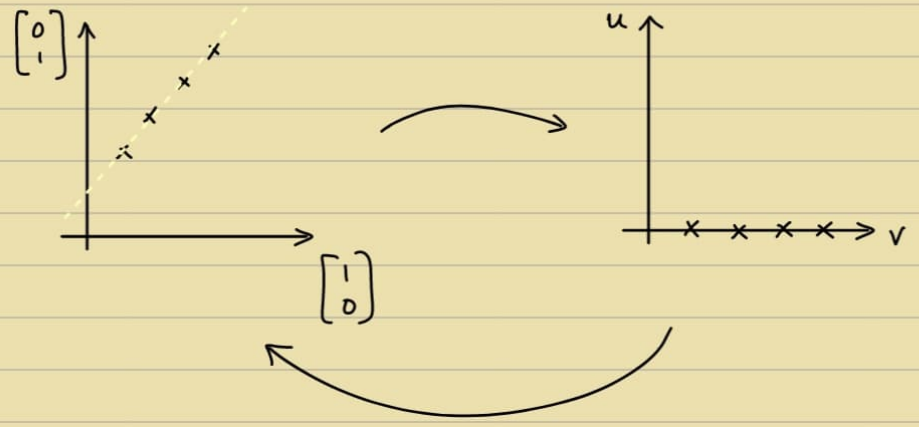
$X_i' = W \cdot X_i \qquad \mathbb{R}^{d'}$

$\downarrow$

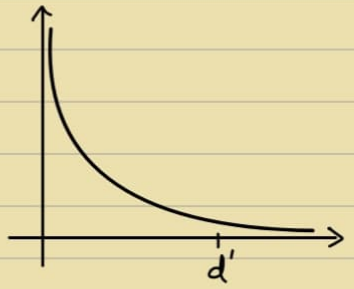
$d' \times d$, E. vector of $\Sigma = \frac{1}{N} \sum_{i=1}^{N} [x_i - \mu][x_i - \mu]^T$

$\mu = \frac{1}{N} \cdot \sum_{i=1}^{N} x_i$

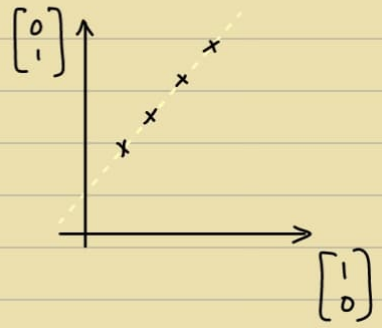
Dimensionality Reduction $\Longrightarrow$ Compression



$d' \times d \Rightarrow d' \times d'$ ∴ Possible, but variations around the line are lost (tiny)



Eigenvalues



$\alpha : u^T x_i$

Minimise $-u^T \Sigma u$ S.T. $u^T u = 1$

Maximise $u^T \Sigma u$ S.T. $u^T u = 1$

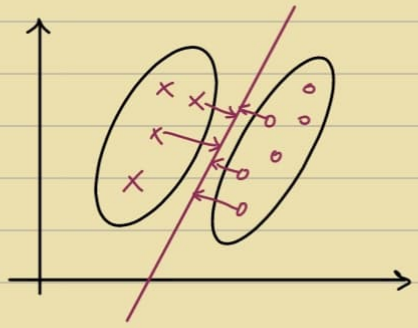
PCA : KL Transform - Basis changes [Unsupervised]

DCT : Basis is Static - LDA [Supervised]

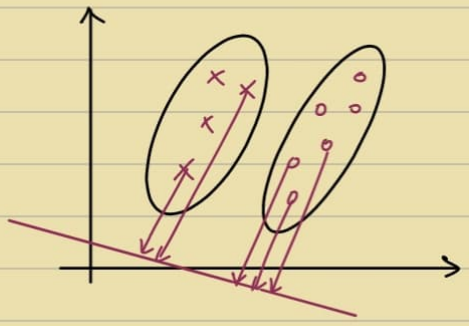
Linear : PCA ; $x' = Wx$

Non-linear : Autoencoder NN ; $x' = f(W, x)$

PCA : Unsupervised



When performing DR



When performing Classification

- DR on Manifolds : LLE , ISOMAP
- Why DR ?
  (i) Less parameters
  (ii) Less Compute $\Longrightarrow$ Faster
  (iii) Better Visualization
  (iv) No Overfitting

- Non-Linear DR : [Autoencoder]

$x \longrightarrow$ Encoder $\longrightarrow z \longrightarrow$ Decoder $\longrightarrow x'$

z : Latent Space

[Visualization : t-SNE]

→ Low Rank Approximation :

- SVD

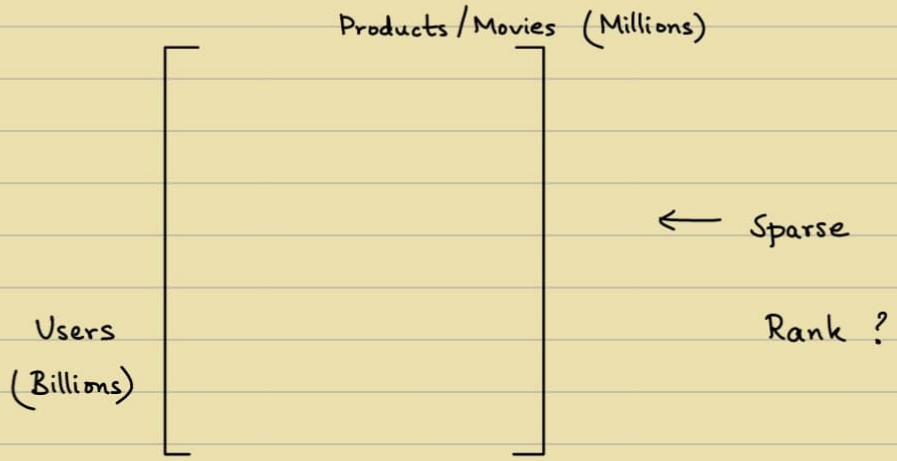
$$A_{m \times n} = \begin{bmatrix} U_k \end{bmatrix} \begin{bmatrix} D_k \end{bmatrix} \begin{bmatrix} V_k \end{bmatrix}$$

$U_{m \times n} \rightarrow U_{k\,m \times k}$

$D_{n \times n} \rightarrow D_{k\,k \times k}$

$V^T_{n \times n} \rightarrow V^T_{k\,k \times n}$

- Low Rank Matrices :



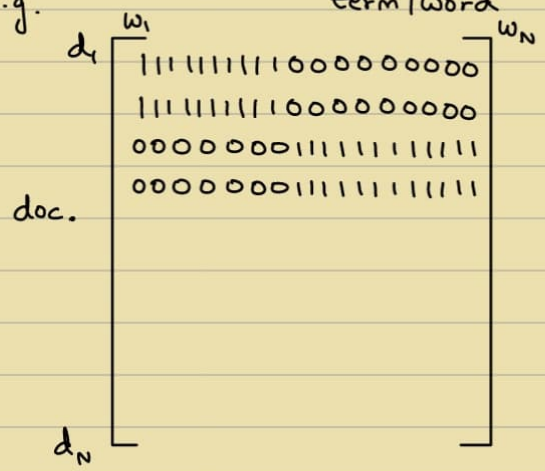
If all Users identical ⇒ Rank = 1
(OR) all Movies identical ⇒ Rank = 1

If all Users Random ⇒ Rank = min (users , movies)
(OR) all Movies Random ⇒ Rank = min (users , movies)

↳ Both are impractical , Actual rank - somewhere in blw

Customer segmentation — Not billions

e.g.



Legal / Medical

Rank = 2
If not sorted , Rank = 2 only
much lower than N or d

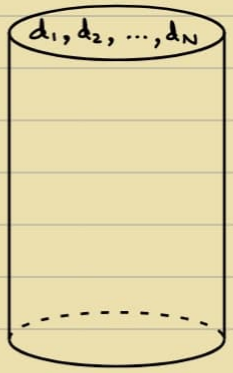
U_k : Any vector of size billion can be projected onto k dimensions
∴ Can find Similar movies
∴ Can recommend

Similarly for V^T : Million dimensions (Movie) ⟶ k dimensions
↳ Very low compared to Million

In the latent space, Problem is well behaved

- Synonymy & Polysemy
  - Polysemy → One word has diff. meaning

One - Hot Encoding



Query document : $d_q \rightarrow k$ - dimension

Diagonal : Sparse $\Longrightarrow$ Rank = 0

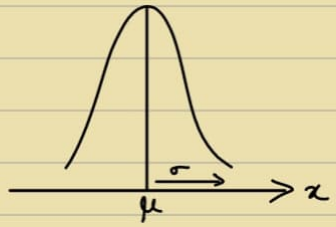
$\therefore L_0$ is best, but dangerous $\because$ We don't know how to use
$\therefore$ Better option : L_1

5/2

→ Normal distribution :

- $x \in \mathbb{R}$, $N(\mu, \sigma^2)$

$$P(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi \cdot \sigma^2}} \cdot e^{\frac{-(x-\mu)^2}{2\sigma^2}}$$



- $x \in \mathbb{R}^d$, $N(\mu, \Sigma)$

$$P(x|\mu, \Sigma) = \frac{1}{(2\pi)^{d/2} \cdot |\Sigma|^{1/2}} \cdot e^{-\frac{1}{2}\left([x-\mu]^T \Sigma^{-1} [x-\mu]\right)}$$

$$\Sigma = \frac{1}{N} \sum_{i=1}^{N} [x_i - \mu][x_i - \mu]^T \longleftarrow \text{Symmetric, PSD}$$

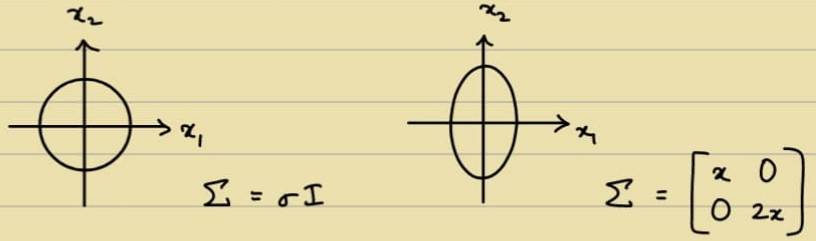
$$z^T \Sigma z \geq 0$$

PSD ?

$$z^T \frac{1}{N} \sum_{i=1}^{N} [x_i - \mu][x_i - \mu]^T z$$

$$= z^T [x_i - \mu][x_i - \mu]^T z \qquad \therefore \text{PSD}$$

- Covariance Matrix :



- Bayes Rule :

$$P(x, \omega_i) = P(x|\omega_i) \cdot P(\omega_i)$$
$$= P(\omega_i|x) \cdot P(x)$$

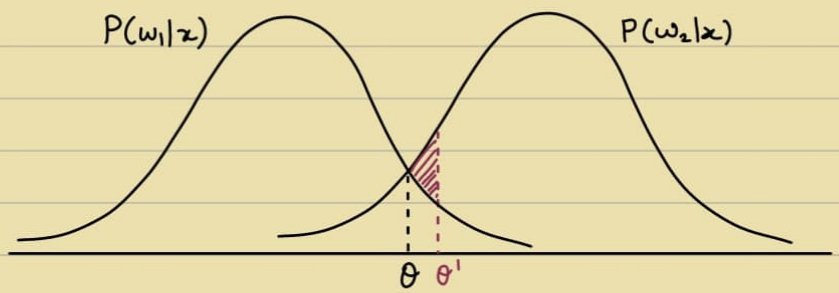
$$P(\omega_i|x) = \frac{P(x|\omega_i) \cdot P(\omega_i)}{P(x)}$$

$$= \frac{P(x|\omega_i) \cdot P(\omega_i)}{\sum_{j=1}^{c} P(x|\omega_j) \cdot P(\omega_j)}$$

$P(\omega_i|x)$: Posterior
$P(\omega_i)$: Prior
$P(x|\omega_i)$: Likelihood

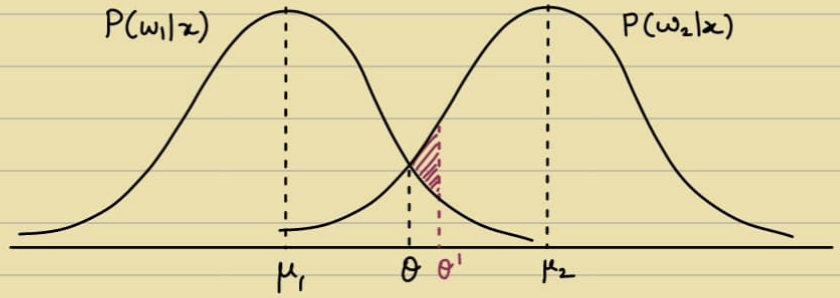
Posterior $\propto$ Prior $\times$ Likelihood

$P(\omega_1|x) \gtrless P(\omega_2|x)$ ← Better Classifier compared to $P(\omega_1) \gtrless P(\omega_2)$
$\Rightarrow P(x|\omega_1) \cdot P(\omega_1) \gtrless P(x|\omega_2) \cdot P(\omega_2)$



Decide as ω_1 if $x \leq \theta$, else ω_2

Note : Bayesian Decision – Optimal , Best Classifer (But Given distribution)
But not used everywhere ∵ Not all data is Gaussian



$\sigma_1 = \sigma_2$

$\theta = \frac{\mu_1 + \mu_2}{2}$

$$(x-\mu_1)^2 \lesseqgtr (x-\mu_2)^2$$

Multivariate gives line instead of Point.

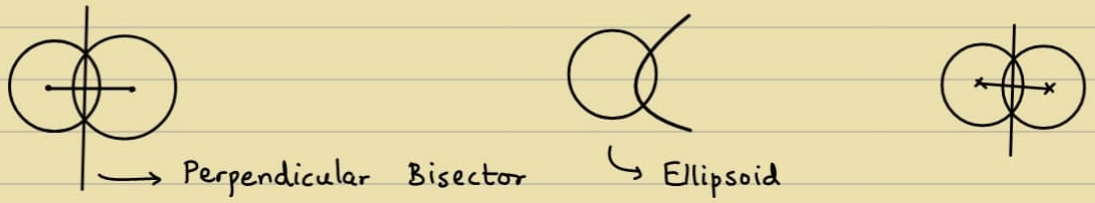
$$e^{\frac{1}{2}[x-\mu_1]^T \Sigma^{-1}[x-\mu_1]} \gtreqless e^{\frac{1}{2}[x-\mu_2]^T \Sigma^{-1}[x-\mu_2]}$$

$$\Rightarrow [x-\mu_1]^T \Sigma^{-1}[x-\mu_1] \gtreqless [x-\mu_2]^T \Sigma^{-1}[x-\mu_2]$$

$$\Rightarrow x^T\omega_1 + \theta_1 \gtreqless x^T\omega_2 + \theta_2$$

$$x^T\omega + \theta \gtreqless 0$$

$\hookrightarrow \text{sign}(\omega^T x)$



→ Perpendicular Bisector ↳ Ellipsoid

$$X^T\Sigma^{-1}X - 2X^T\Sigma^{-1}\mu_1 + \underbrace{\mu_1^T\Sigma^{-1}\mu_1}_{\theta_1} \gtreqless X^T\Sigma^{-1}X - 2X^T\Sigma^{-1}\mu_2 + \underbrace{\mu_2^T\Sigma^{-1}\mu_2}_{\theta_2}$$

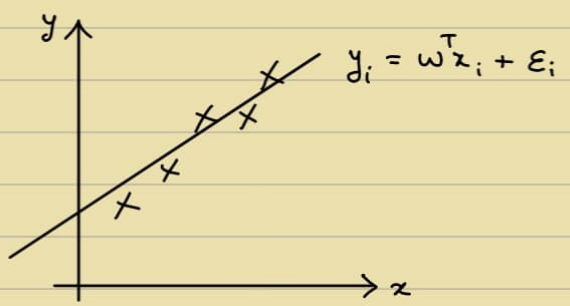
$$\Rightarrow 2X^T\Sigma^{-1}[\mu_1-\mu_2] + \underbrace{(\theta_1-\theta_2)}_{\theta} \gtreqless 0$$

- LR :

$$y = \omega^T x + \varepsilon_i$$

$$L(\omega) = \prod_{i=1}^{N} \frac{1}{\sigma\sqrt{2\pi}} \cdot e^{\frac{-(y_i-\omega^T x)^2}{2\sigma^2}}$$

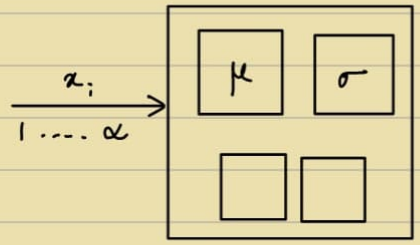
Max $L(\omega)$ ⇒ Max $\log(L(\omega))$



$$\ell(\omega) = K \cdot \frac{1}{2} \cdot \sum_{i=1}^{N} (y_i - \omega^T x)^2 \qquad [K : \text{Const.}]$$

$$\min_{\omega} J(\omega) = \min \sum_{i=1}^{N} (y_i - \omega^T x)^2 \longleftarrow \text{MSE}$$

- Streaming data :



① $\mu = x_1$

② $\mu = \frac{\mu + x_2}{2}$

③ $\mu = \frac{2\mu + x_3}{3}$

$$\mu_{N+1} = \frac{(\mu_N \times N) + (1 + x_{N+1})}{N+1}$$

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^{N} (x_i - \mu)^2$$

$$= \frac{1}{N} \sum_{i=1}^{N} (x_i^2 - 2\mu x_i + \mu^2)$$

- Feature Selection : [Hard]

$$x' = WX$$

↳ Size : 1000

Want to select 10 best

Dimensionality Reduction : Cannot be used ∵ Changes features

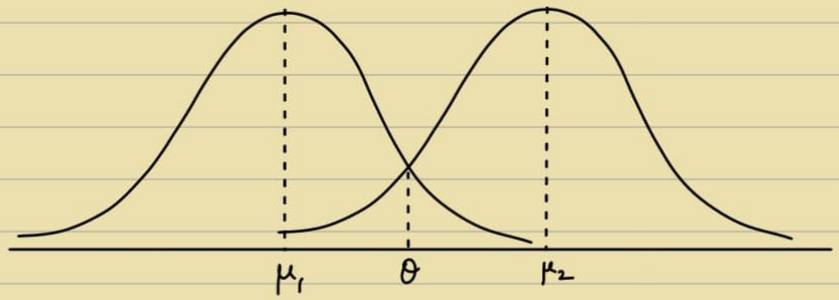
e.g. Lasso Regularization

Generally done : Greedy Algo.

Rank each feature based on $\frac{\mu_A^j - \mu_B^j}{(\sigma_A^j)^2 + (\sigma_B^j)^2}$

[KL Distribution ?]

9/2



$D = \{x_1, x_2, \dots, x_n\}$

Q) Compute Mean

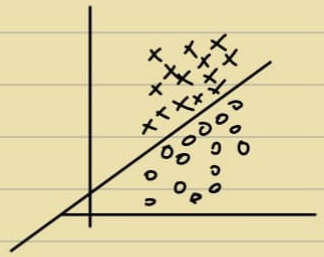
|||

Q) Which distribution

$N(\mu, \sigma^2)$ - 1D

$N(\mu, \Sigma)$ - Multivariable

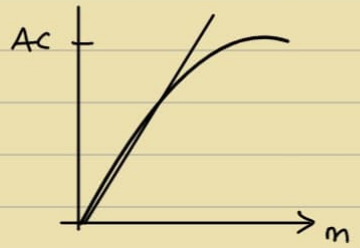
$x_i \in \mathbb{R}^d$



Note: PAC

(Probability Approximate Learning)

$m > f\left(\frac{1}{\varepsilon}, \frac{1}{\delta}\right)$



$$\text{Posterior} = \frac{\text{Prior} \times \text{Likelihood}}{\underbrace{\text{Evidence}}}$$

↳ const. @ Comparison

$P(\omega_1|x) \gtrless P(\omega_2|x)$

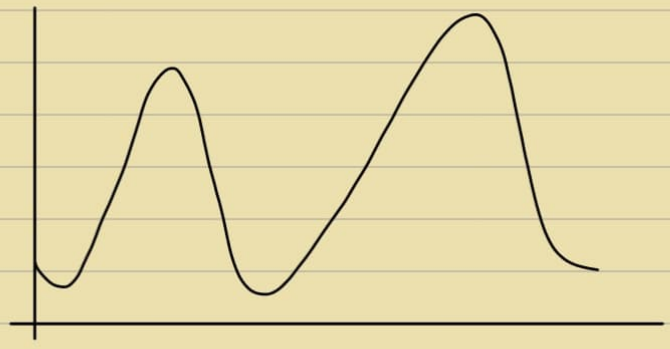
$\Sigma_i = \sigma^2 I$

$\Sigma_i = \Sigma$

Linear

Quadratic

→ Multimodal Curves:



	P_H	P_T	
C_1	0.5	0.5	– 0.0312
C_2	0.7	0.3	– 0.01323
C_3	0.2	0.8	– 0.02048

Came from $N(\mu_1, \sigma_1^2)$ & $N(\mu_2, \sigma_2^2)$

- MLE – $\arg\max_{\text{model}} P(\text{Data} \mid \text{Model})$

$$\theta^* = \arg\max_{\theta} P(X \mid \theta)$$

Q) Given $X = \{x_1, x_2, \ldots x_n\}$
Find μ, σ^2 corresponding to N

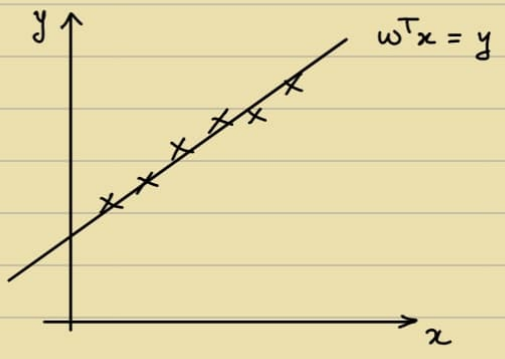
Log-likelihood — Easier

e.g. $P(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{\frac{-(x-\mu)^2}{2\sigma^2}}$

Apply MLE,

$$\mu = \frac{1}{N} \sum_{i=1}^{N} x_i \qquad \sigma^2 = \frac{1}{N} \sum_{i=1}^{N} (x_i - \mu)^2$$

For LR:
MLE $\Rightarrow$ MSE



$y = w^T x + \varepsilon_i$
$\varepsilon_i \hookrightarrow N(\mu, \sigma^2)$

GMM – Complex Multi-modal distribution

$$\sum_{k=1}^{k} \pi_k \cdot N(\mu_k, \sigma_k^2)$$

π_k → Mixing
$N(\mu_k, \sigma_k^2)$ → θ

If we know mixing, we can find θ.
If we know θ, we can find mixing.

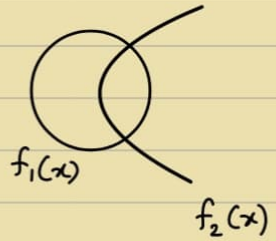
$$\theta = \{\mu, \sigma^2\}$$

Fixed Point Iteration - Proof of convergence (not in course's scope)

Risk factor : $\lambda_1 P(\omega_1|x) \geq \lambda_2 P(\omega_2|x)$ - Decide as ω_1, else otherwise

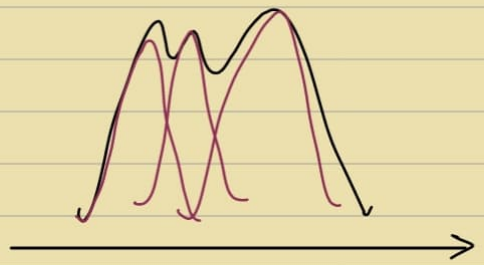
→ K-Means : Clustering Algo

→ K-Means, Naive Bayes and Logistic Regression :

- 

$\max f_i(x)$

- Component distribution :



- GMM Algo :

$$\gamma_k(x) \in [0,1]$$

$$\pi_j = \frac{1}{N} \sum_{i=1}^{N} \gamma_j(x_i)$$

↳ % of Samples of the j^{th} component to the total

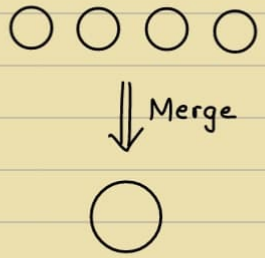
Assume C_k a set with $\gamma_k(x_i) = 1 \implies C_k \subseteq X$
↳ $\{x_1, x_2, \ldots, x_N\}$

μ_j = mean of C_j^{th} Subset
Σ_j = Covariance of C_j^{th} Subset

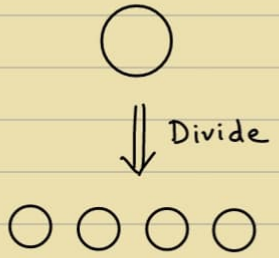
$$X = C_1 \cup C_2 \cup \ldots \cup C_k \quad \text{S.T.} \quad C_i \cap C_j = \phi$$

if μ_j & Σ_j are binary $\Rightarrow$ K-Means

(i) Agglomerative :



(ii) Divisive



e.g. Create MST and remove longest edge.

e.g. Hashing - Hash functions
LSH : Locality Sensitive Hashing

ANN : Approximate Nearest Neighbour $\Rightarrow$ Better than KNN

- <u>K-Means</u> :

$$\min_i \sum_i \sum_k \| x_i - \mu_k \|_2^2 \quad , \quad x_i \in C_k$$

μ_k : Mean of cluster C_k

Randomly partition of set into K Subsets
Compute means of these subsets as $\mu_1, \mu_2, \ldots, \mu_k$
Reassign every sample to the set where mean is closest
Repeat above 2.

$$J(\ ) = \sum_i \sum_k \| x_i - \mu_k \|^2$$

e.g. $C_1 = \{2, 5, 3, 16, 5\}$ $\quad \mu_1 = 8.2$ $\quad C_1 = \{1, 2, 3, 4, 5\}$
$C_2 = \{1, 4, 18, 22, 20\}$ $\quad \mu_2 = 13$ $\quad C_2 = \{15, 16, 18, 20, 22\}$

e.g. $C_1 = \{A(1,1), B(1.5,2), C(3,4)\} \Rightarrow \mu_1 = (1.83, 2.33)$

$C_2 = \{D(5,7), E(3.5,5), F(4.5,5)\} \Rightarrow \mu_2 = (4.33, 5.67)$

$\Downarrow$

$C_1 = \{A(1,1), C(3,4), D(5,7)\} \Rightarrow \mu_1 = (3,4)$

$C_2 = \{B(1.5,2), E(3.5,5), F(4.5,5)\} \Rightarrow \mu_2 = (3.167, 4)$

- Naive Bayes:

$$P(Y|X) = \frac{P(X|Y) \cdot P(Y)}{P(X)}$$

$X_1, X_2, \ldots X_d \qquad y \in \{0,1\}$

$\quad \hookrightarrow \{0,1,2..\}$

We assume features are conditionally independent given class y

$$P(x_1, x_2, \ldots, x_d \mid y) = \prod_{j=1}^{d} P(x_j|y)$$

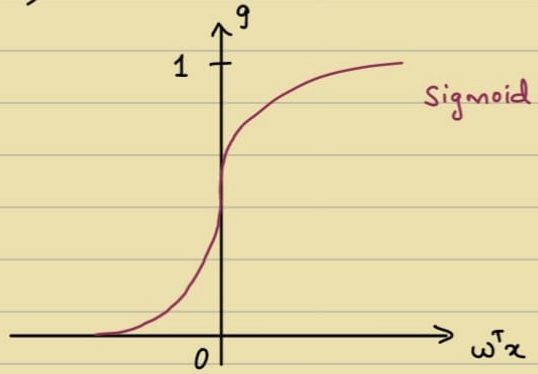
$$\Rightarrow P(x^1 x^2 \ldots x^d | Y) = \prod_{i=1}^{d} P(x^i|Y)$$

$P(x_1, x_2, \ldots, x_d) \leftarrow$ Not Practical

$\therefore$ Use conditional independence - Naive Independence

- Logistic Regression:
  - Classification Algo
  - Logistic function / Sigmoid

$\text{sign}(\omega^T x)$



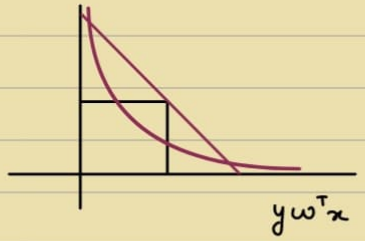
$$g(x) = \frac{1}{1+e^{-x}}$$

$$1 - \frac{1}{1+e^{-x}} = \frac{1}{1+e^{x}}$$

$$P(y|x) = g(\omega^T x)^y \left(1 - g(\omega^T x)\right)^{1-y}$$

Binary Cross Entropy : $-[y \log p + (1-y) \cdot \log(1-p)]$ $\leftarrow$ MLE

LR with $y \in \{-1, 1\}$ → Equation becomes compact



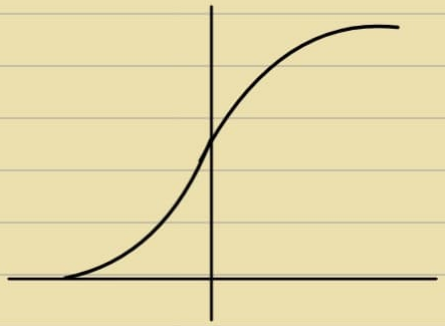
$\omega^T x > 0 \quad y = +1$

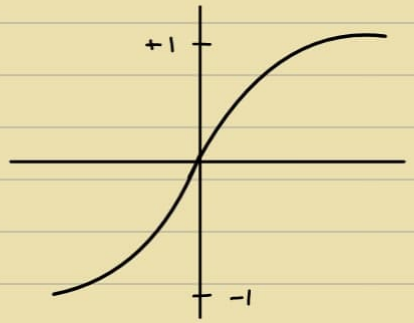
$\omega^T x \leq 0 \quad y = -1$

$y(\omega^T x) \geq 0$

$\arg\max_i p(y = i \mid x)$ ← Softmax

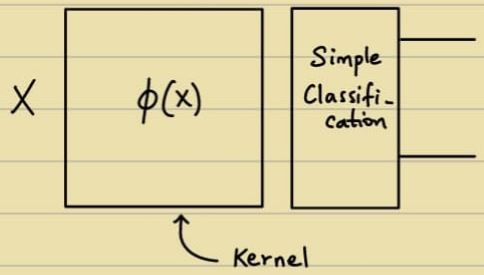
$$P(y = c \mid x ; \omega_1, \omega_2, \omega_k) = \frac{e^{\omega_c{}^T x}}{\sum_{i=1}^{k} e^{\omega_i{}^T x}}$$





16/2

→ SVMs :



Feature extraction

Feature Transformation

(1) minimise no. of misclassification

(2) Find a line S.T.

Constraints X is on +ve side

O is on -ve side

(Constr. set)

Hinge loss / SVM loss :

$L = \max(0, 1 - yf(x))$

* Maximise Margin b/w 2 lines S.T.

All samples are atleast unit dist.

$x^T\omega + b = 1$

$$\text{sign}(\omega^T x + b)$$

$$f(x_i) = x_i \cdot \omega + b$$

$$\therefore \; x_i \cdot \omega + b \geq +1 \quad \text{for} \quad y_i = +1$$

$$x_i \cdot \omega + b \leq -1 \quad \text{for} \quad y_i = -1$$

$$x_i \cdot \omega \equiv \omega^T x$$

Origin to $ax + by + c = 0$: $c / \sqrt{a^2 + b^2}$

for $\omega^T x + b = +1$: $\frac{1-b}{\|\omega\|}$

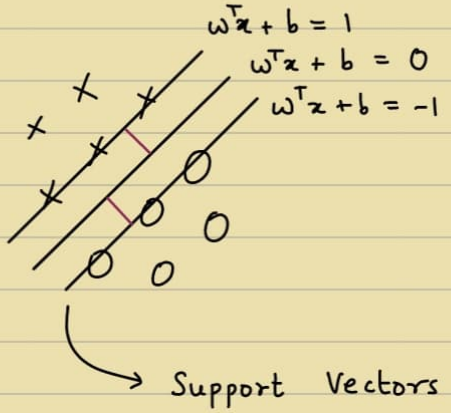
$\omega^T x + b = -1$: $\frac{-1-b}{\|\omega\|}$

$$\text{Margin} = \frac{2}{\|\omega\|} = \frac{2}{\omega^T \omega}$$

$$y_i(\omega \cdot x_i + b) - 1 \geq 0 \quad \forall i$$

Objective : $\min. \frac{1}{2} \omega^T \omega$ S.T. $y_i(\omega \cdot x_i + b) - 1 \geq 0 \quad \forall i$

$y_i \in \{-1, 1\}$



→ Support Vectors (α is non-zero)

α is zero for non-support vectors

Dual : $J_d(\alpha) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j x_i^T x_j$

$$\Rightarrow \omega = \sum_{i=1}^{N} \alpha_i y_i x_i$$

$$b = \frac{1}{|s|} \sum_{i \in s} (y_i - \omega^T x_i)$$

During Testing :

$$\text{sign}\left(\sum_i \alpha_i y_i x_i^T x + b \right)$$

Linear non-seperable ?

(XOR Problem ?)

Hard Margin ✗

Soft Margin with small violation ✓

- Add penalty ξ_i & Add to objective

$$x_i \cdot w + b \geq +1 - \xi_i \quad \text{for} \quad y_i = +1$$
$$x_i \cdot w + b \leq -1 + \xi_i \quad \text{for} \quad y_i = -1$$
$$\xi_i \geq 0 \quad \forall i$$

Objective :

$$\frac{1}{2} w^T w + C \sum_i \xi_i \qquad (L_1) \qquad L_1 : \text{Sparse}$$

$$\frac{1}{2} w^T w + \frac{C}{2} \sum_i \xi_i^2 \qquad (L_2)$$

- Kernels :

Dot - Product is scalar independent of Dimension.

$$p = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \qquad p^T q = 4 \Leftarrow p_1 q_1 + p_2 q_2$$

$$q = \begin{bmatrix} 2 \\ 2 \end{bmatrix} \qquad \phi(p)^T \phi(q) = ? \qquad \text{Without knowing } \phi$$

$$(p^T q)^2 = p_1^2 q_1^2 + p_2^2 q_2^2 + 2 p_1 q_1 p_2 q_2$$

$$= \begin{bmatrix} p_1^2 \\ p_2^2 \\ \sqrt{2} p_1 p_2 \end{bmatrix}^T \begin{bmatrix} q_1^2 \\ q_2^2 \\ \sqrt{2} q_1 q_2 \end{bmatrix}$$

$$\phi : \mathbb{R}^2 \to \mathbb{R}^3 \qquad \phi(p) \qquad \phi(q)$$

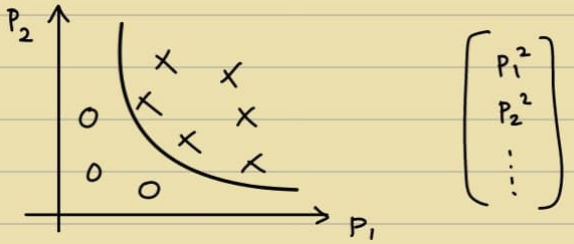
Kernel : $K(p,q) = (p^T q)^d$

$K(p,q) = (1 + p^T q)^d$

$K(p,q) = \exp(-p^T q)$ → ∞ - dimensional space (Sigmoid, RBF, etc)

→ Stong feature mapping

Polynomial $\xrightarrow{\text{Kernel}}$ Linear Seperable



$$\begin{bmatrix} p_1^2 \\ p_2^2 \\ \vdots \end{bmatrix}$$

Kernel SVM – Heavy Memory while Testing (∵ Carry S.Vectors)
Linear SVM

e.g. $N = 10000$
$|SV| = 100$

Error estimate = ?
Leave One Out (LOO) $\Longrightarrow$ error $\leq \frac{|SV|}{N}$

(Removing non-support vectors does not affect, ∴ $|SV|$ ← Upper bound)

$K : N \times N \longrightarrow$ Symmetric, Positive Semi-definite
$K_{ij} = K(x_i, x_j) \rightarrow (x_i^T x_j)^2$

e.g. Kernel PCA

- Hinge Loss curve → Convex

Big ϕ & Small classifier vs. Small ϕ and Big Classifier

Hinge Loss - Optimal Solution
Logistic Loss $\Longrightarrow$ Hinge Loss

(Primal $\Longrightarrow$ Dual ?)

VC dimension

- C : Rate of misclassification
  ↳ Hyperparameters

- SVM - Binary class classification
  ↳ Long training time

19/2

Q) $\{x_1, x_2, \ldots, x_N\}$

$x_i \in \mathbb{R}^d$

Build the best binary classifier

(1) Dimensionality Reduction

Accuracy - better ?

(2) Normalization

$\|x_i\| = 1$ $\quad \{x\}$ ← Dimension specific

$\mu = 0$ $\quad$ Vector ← usually have subsets

$\sigma = 1$ $\quad$ Set

e.g. $\begin{bmatrix} [p] \\ [q] \\ [r] \end{bmatrix}$

Step - I: $\|p\| = 1$, $\|q\| = 1$, $\|r\| = 1$ $\Rightarrow$ Step - II: $\|x\| = 1$ $\Rightarrow$ Step - III: $\{\text{Set}\}$, $\mu = 0$

- Minimizing only MSE ✗

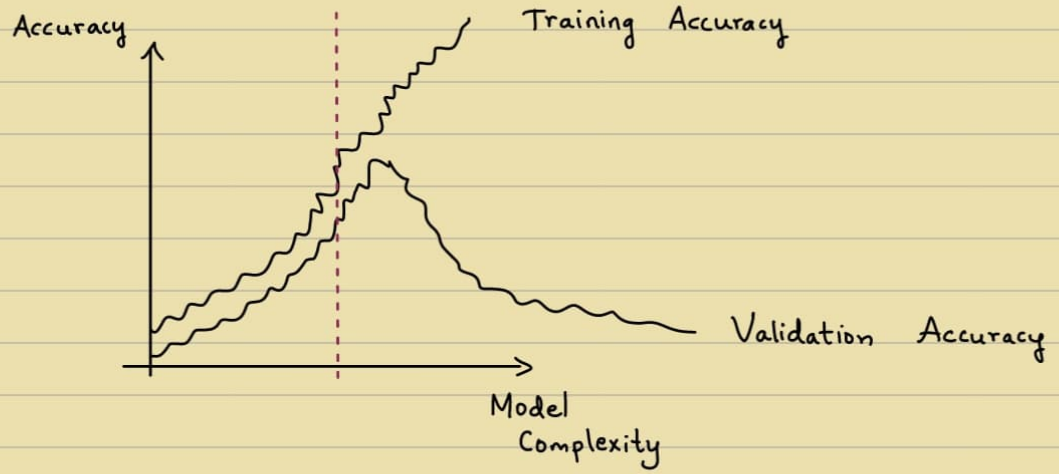
  Minimize (MSE + Complexity of sol.)

  i.e. Loss + Complexity of solution

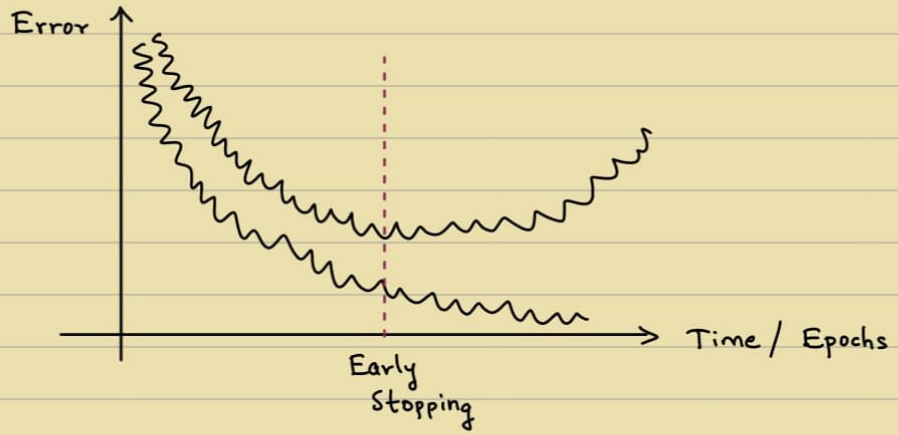
  ∴ Regularisation is important

  Complexity can be degree of polynomial as well.

– Go through all possibilities of model complexities.



$$\omega^{k+1} \leftarrow \omega^{k} - \eta \cdot \frac{\partial L}{\partial \omega^{k}}$$



- Overfit : Modeling error
  i.e. Train Error << Test Error
  Train Accuracy >> Test Error

Avoid Overfitting by
(i) Cross Validation — k subsets : k-1 training & 1 evaluation : k times
  └ Leave One Out (LOO)
(ii) Train with more data
(iii) Remove some features
(iv) Regularisation
(v) Ensembling
(vi) Early Stopping

- Train - Train
  Val - Internal Test Data
  Test - Test

- Bias vs. Variance
  ↳ Error from sensitivity towards data
  ↳ Error from wrong assumption

e.g. KNN - k↑ : Bias ↑ & Variance ↓
k↓ : Bias ↓ & Variance ↑

Decision - Shallow Depth : Bias ↓
Deep Depth : Bias ↑

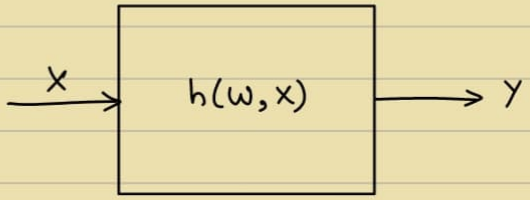
- Parameters vs. Hyperparameters
  - Changes while training
  - ↳ Not changed while training (control how the model learns)

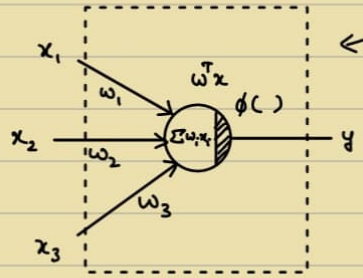
9/3

(x_i, y_i) ← Concept (Data)

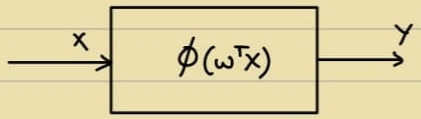
$y : f(x)$ ← Learning

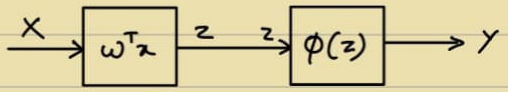
Find the best $h \in H$ that can approximate f on my data D.



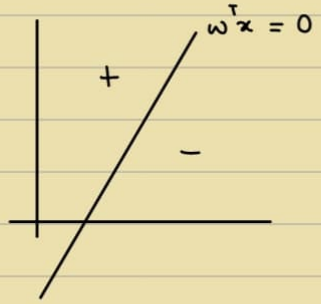


$h \in H$



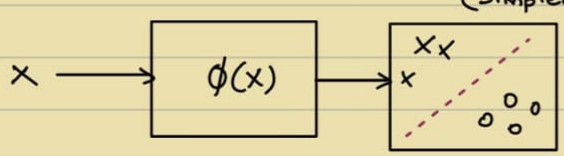


ϕ : sign ( )



Converges only when data is linearly seperable

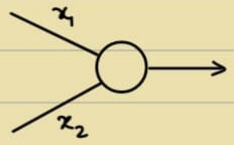
- Ex-OR & Linear Non-seperability



$\phi(x) : x \rightarrow x'$

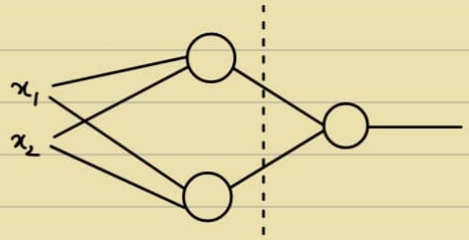
← Kernel ($x = \phi^T \phi$)

← NN



Not enough for XOR

AND / OR ✓



Ex OR

x_1	x_2	A	O	E
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

$\bar{x}_1 x_2 + \bar{x}_2 x_1$

Lines ⟶ Convex Regions
(Overlapping Arbitrary Shapes)

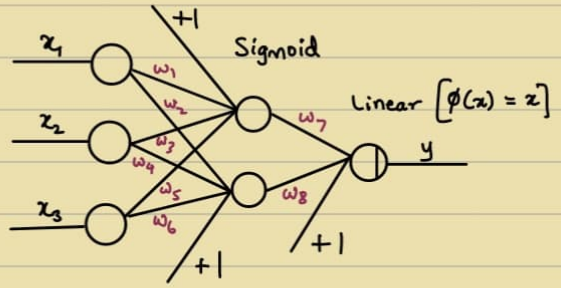
- Multi-Layer Perceptron : (MLP)
  (Fully Connected Layers)
  - Feed-Forward Neural Network

- Activations :

Sign ⟹ Sigmoid (0 to 1)
tanh (-1 to 1)

ReLU - Piece-wise Linear
↳ $\max(0, x)$

9)



$\omega^T x$

$$\begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \\ \omega_0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ 1 \end{bmatrix}$$

$\phi(z) = \frac{1}{1+e^{-z}}$

6 + 2 learnable weights without bias

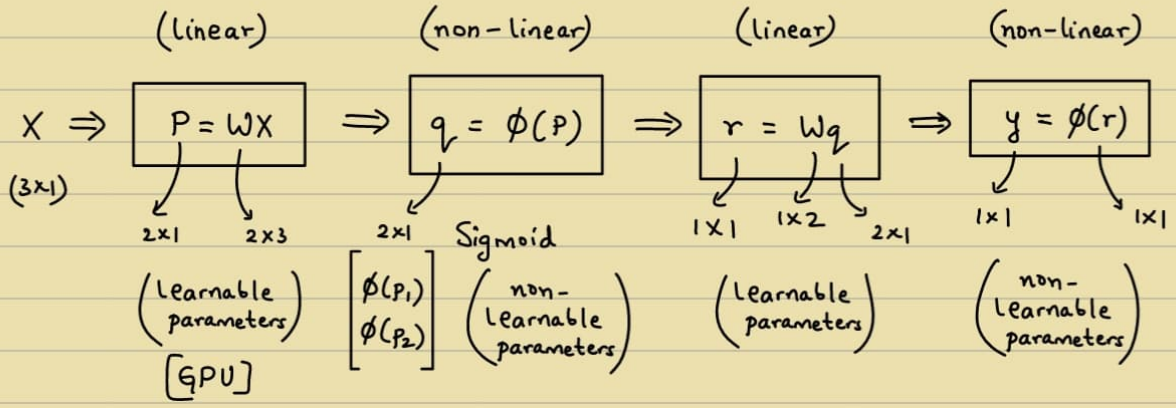
8 + 1 + 1 + 1 learnable weights with bias

$$y = w_7(\ ) + w_8(\ )$$

$$= w_7\left(\frac{1}{1+e^{-(w_1x_1+w_2x_2+w_3x_3)}}\right) + w_8\left(\frac{1}{1+e^{-(w_4x_1+w_5x_2+w_6x_3)}}\right)$$

$$P = \begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$P = WX$$



If :

$$X \Rightarrow \boxed{P = W_1X} \Rightarrow \boxed{q = W_2P} \Rightarrow \boxed{r = W_3q} \Rightarrow r$$

$$y = W_3W_2W_1X$$

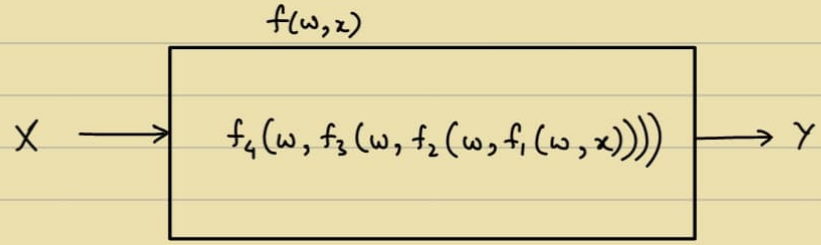
No ReLU ⇒ SLP ∴ Non-Linearity is imp.
(if no ϕ)

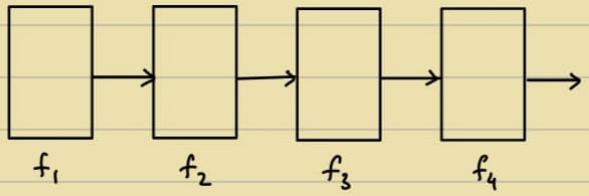
Sigmoid : $\sigma(z) = \frac{1}{1+e^{-z}}$

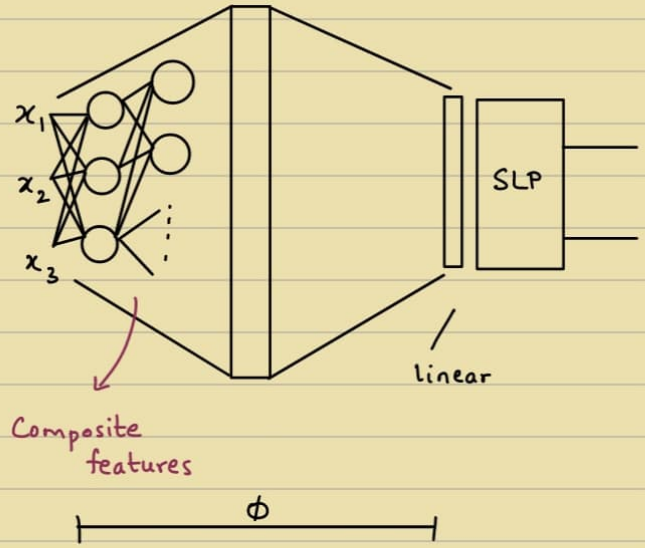
$$\frac{d\,\sigma(z)}{dz} = \sigma(z)[1-\sigma(z)] \qquad [\text{Similarly, } \tanh(z)]$$

Comes handy when backpropagation, Grad. Descent, etc.

Why hidden layers ?
Why activation functions ?







UAT : Any function can be approximated.

- MLP for Regression :

  $y = \phi(x)$

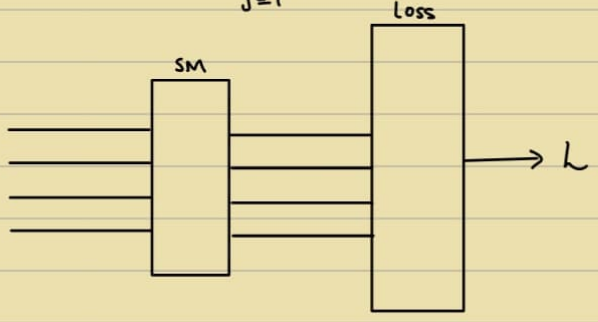
  $y \in \mathbb{R}$

  Linear activations are generally used

Softmax layer :

Convert numbers into probabilities

$$\text{Softmax} : \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$$



- Backprop :

Initialize weights randomly

Compute x_{p+1}

$$L = \frac{1}{N} \sum_{i=1}^{N} L(x_{p+1}^{i}, y_i) \quad \text{full batch}$$

Then, Adjust :

$$\theta^{k+1} \leftarrow \theta^{k} - \Delta\theta$$

$$\theta^{k+1} \leftarrow \theta^{k} - \eta \frac{\partial L}{\partial \theta}$$

} Loop

$\frac{\partial L}{\partial \theta}$ ← Expensive

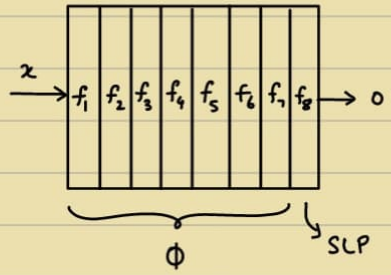
Next class : Backpropagation

$\frac{\partial L}{\partial w_1}$? for φ Loss : compare y_i with $\hat{y}_i$

$y \sim \hat{y}$

12/3

Concetanted blocks

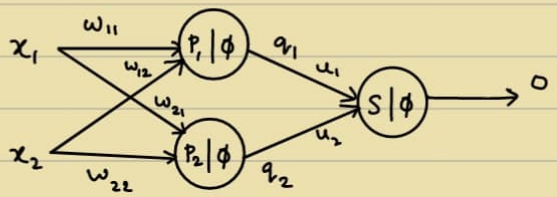


$$o = f_8(f_7(\ldots.(f_1(x))\ldots)$$

→ Semantic

e.g. $f = \frac{1}{1+e^{-z}}$

e.g.



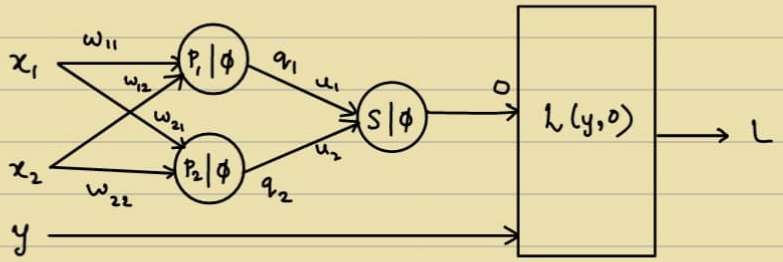
$$\begin{bmatrix} P_1 \\ P_2 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \qquad \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} = \begin{bmatrix} \phi(P_1) \\ \phi(P_2) \end{bmatrix}$$

$$s = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}^T \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}$$

$p = Wx$

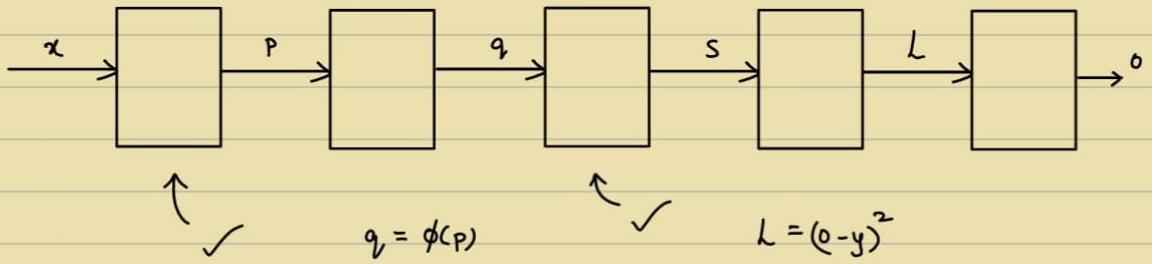
$q = \phi(p)$ $\qquad$ $o = \phi(s)$

$s = u^T x$ $\qquad$ $L = L(y, o) = (o - y)^2$



Softmax layer : $\dfrac{e^-}{\sum e^-}$

Learnable parameters : weights



$p = Wx$ $\qquad$ $s = u^T x$

As $\quad w^{k+1} \leftarrow w^k - \eta \dfrac{\partial L}{\partial w^k}$

$$\frac{\partial L}{\partial u_1} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s} \cdot \frac{\partial s}{\partial u_1}$$

$$= 2(o-y) \cdot \phi'(s) \cdot q_1$$

Reusable when $\dfrac{\partial L}{\partial u_2}$

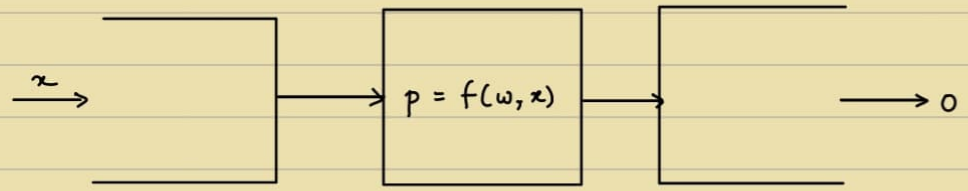
$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s} \cdot \frac{\partial s}{\partial q_1} \cdot \frac{\partial q_1}{\partial p_1} \cdot \frac{\partial p_1}{\partial w_{11}}$$

$$= 2(o-y) \cdot \phi'(s) \cdot u_1 \cdot \phi'(p_1) \cdot x_1$$

Reusable when $\dfrac{\partial L}{\partial w_{12}}$

Updating weights - learning

Long chain of multiplication ← Vanishing Gradients / Exploding Gradients



$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial x} \quad \text{(Layer)}$$

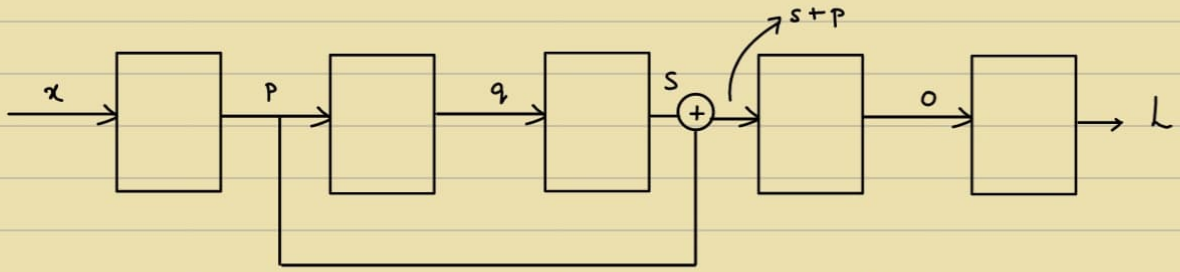
$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial w}$$

H·W

Define custom learnable sigmoid layer in PyTorch and train an MLP

$w \uparrow \Rightarrow$ Steeper

Q)



↳ Skip connection / Residual connection

Find $\frac{\partial L}{\partial w}$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s'} \cdot \frac{\partial s'}{\partial q} \cdot \frac{\partial q}{\partial p} \cdot \frac{\partial p}{\partial w} + \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial r} \cdot \frac{\partial r}{\partial p} \cdot \frac{\partial p}{\partial w}$$

$$\left\{ \begin{array}{l|l} p = Wx & o = \phi(s+p) \\ q = \phi(p) & L = L(y,o) = (o-y)^2 \\ s = u^T x & \end{array} \right\}$$

$$s + p = u^T x + Wx = (u^T + W)\, x$$

$$\frac{\partial s'}{\partial q} = \frac{\partial s'}{\partial s} \cdot \frac{\partial s}{\partial q}$$

(Solves Vanishing gradient problem)

Then, $w^{n+1} \leftarrow w^n - \eta \frac{\partial L}{\partial w}$

Next Class : Build on Backprop. (Grad. Descent)

23/3

Neural Network : Approximation of f

e.g. $y = 4x^2 + 2x$

$\Rightarrow y = f(x)$

Problems :

① Problem Abstraction

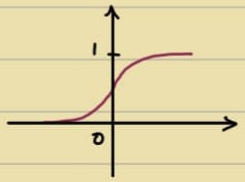
② Choice of Architecture

③ Training ← Backpropagation

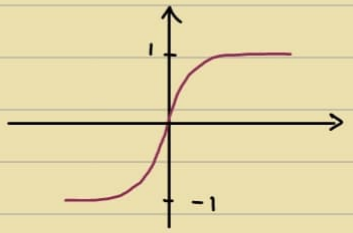
forward pass : Compute loss

Backward pass : Update Weights

~ 85% – Baseline

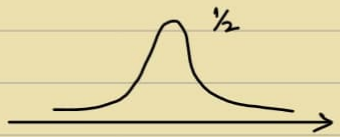


Sigmoid (f_1)

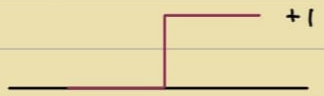


tanh (f_2)

ReLU (f_3)

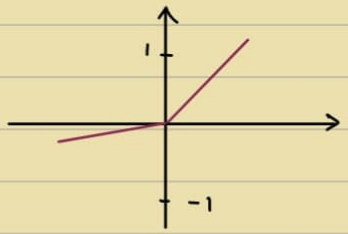


f_1'



f_3'

Leaky ReLU :



f_3

It's not about implementing BP

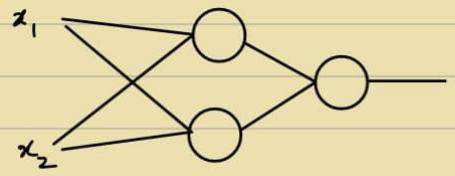
Can you use BP and train a NN ?

Subgradient ?

- Upgrading BP:

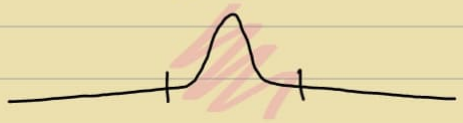
(1) Better Initialisation

if all weights are equal or zero



If both weights are equal, then outputs will be same. We want them to complement each other.

Random, but weighted sum in this region.



Xavior initialisation

Cunningham initialisation

(2) Gradient Estimates

We don't use loss, we use $\frac{\partial L}{\partial o}$ (Gradient of loss)

Derivative of Loss - Exact or Approximate ?

We're looking for the direction of the gradient and not the magnitude - Estimate is fine.

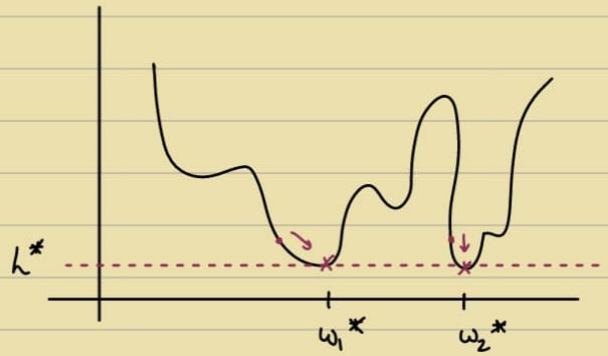
Reasonable estimate in $1/1000^{th}$ time ✓ $\quad w^{(k+1)} \leftarrow w^{(k)} - \eta \cdot \frac{\partial L}{\partial w^{(k)}}$

Whole batch - Very Expensive

Single Sample - Very efficient but noisy

Stochastic Mini-batch GD - Balance b/w both

e.g. Loss function



Multiple Local Minima

↳ Not a problem

Same performance (approx)

Local Minima — Derivatives of all weights / params. = 0 @ point
↳ Probability is low

Epoch vs. Iteration
Learning Rate — Schedulars

Momentum : $\theta^{k+1} \leftarrow \theta^k - \Delta\theta$

$$\Delta\theta = \eta \frac{\partial L}{\partial \theta} + \gamma \Delta\theta^{t-1}$$

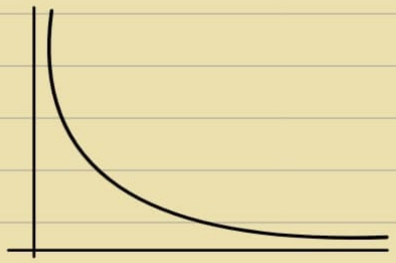
↳ Additional term

(3) Regularization :

Data Augmentation — Params to Data ratio ↑
Add noise — Inputs, Outputs, weights
Loss function — L_1, L_2 regularization

- Early stopping — stop when val. acc. ↑

Will loss = 0 after long time training ? No
Model Complexity
Data Noise



Won't increase and non-zero
Always decreasing.

26/3

- Choose good initialisation
  i.e. Take Pre-trained network

Fine-tune : domain exclusive training

- Normalisation — Batch Norm
  ↳ Layer → $\hat{x} = f(w, x)$

- Dropout

- For quadratic loss function, GD Convergence in 1 Step is possible
  But, we are truncating from taylor series

  When $f(x) \rightarrow$ linear $\Rightarrow$ Only Grad. is enough
  When $f(x) \rightarrow$ Quad $\Rightarrow$ Grad. & Hessian is req.

  But we are still approximating, $\therefore$ Not exact

  Optimal $\eta = \frac{\|\nabla J\|^2}{\nabla J^T H \nabla J}$ $\rightarrow$ Calc. is memory expensive.
  $[H : d \times d]$

  Memory vs. Time Trade off

- $J(w^{k+1}) = J(w^k) + s^T \nabla J + \frac{1}{2} s^T H s$

  $\frac{d}{ds} \overset{\text{set}}{=} 0 \Rightarrow$ Newton Method

  $w^{k+1} = w^k - (H + \alpha I)^{-1} \nabla J$

  $(H + \alpha I)$ $\because$ Always Invertible
  But Inversion is costly $O(d^3)$

  $\alpha \uparrow \Rightarrow$ G.D.

  Steepest GD : Find steep Grad. direction

- Momentum :
  η gives diff. η's for diff. values

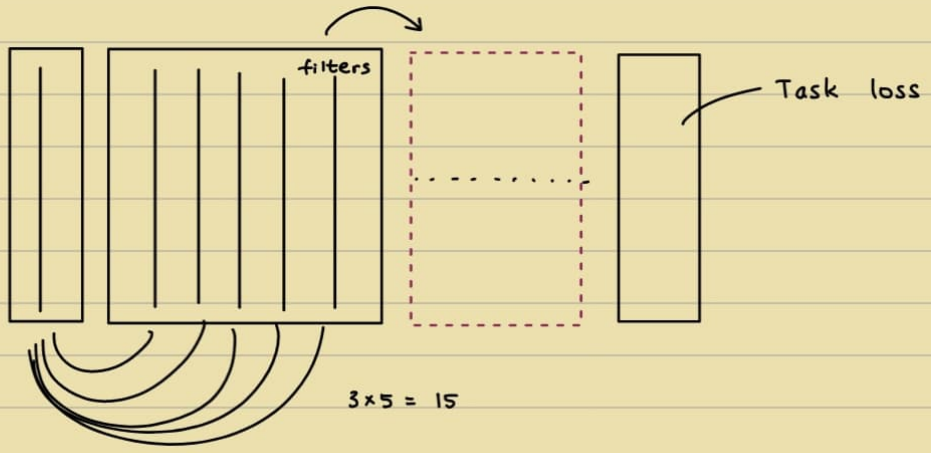
  Adam : Momentum + Adaptive Scaling
  (+ Bias correction)

2/4

$\rightarrow$ CNN :

Convolution : $f * g = \int_{-\infty}^{\infty} f(x) \cdot g(t-x)\, dx$

In CNN $\Rightarrow$ Correlation, i.e. Dot Product with Padding



No. of weights in independent of size

Whereas in MLP :

$n \times n \longrightarrow n^2$ weights to learn

But in CNN,

Independent of n

No Padding ⇒ Output size decreases

- Window size

Pooling

Stride

Pad

Flattening

AlexNet - Oversaturated features

Then, Deeper & Lighter

2 3×3 filters better than 1 5×5 filters

1 × 1 Kernel Size is used for dimentionality Reduction.

6/4

→ More on CNNs :

1 D Conv. Kernels / Filters

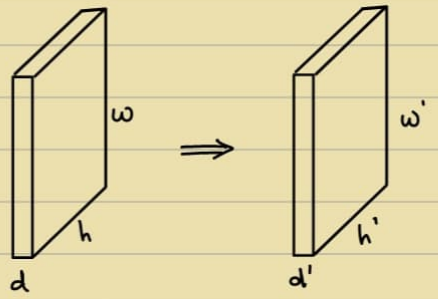
2D Conv. Feature Maps

3D Conv.

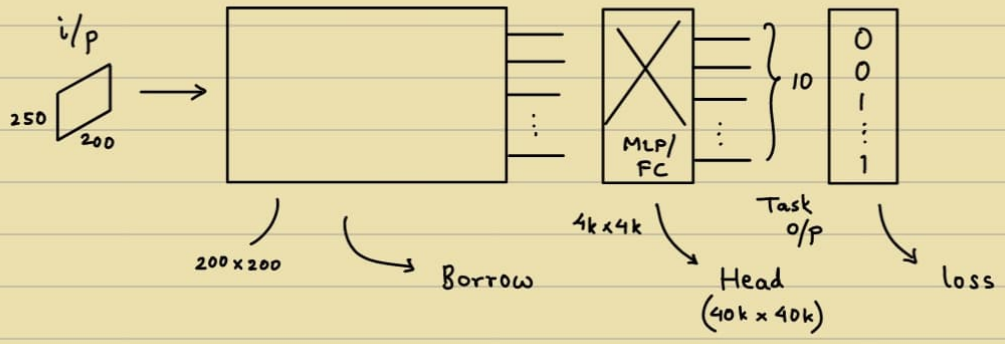
Pad

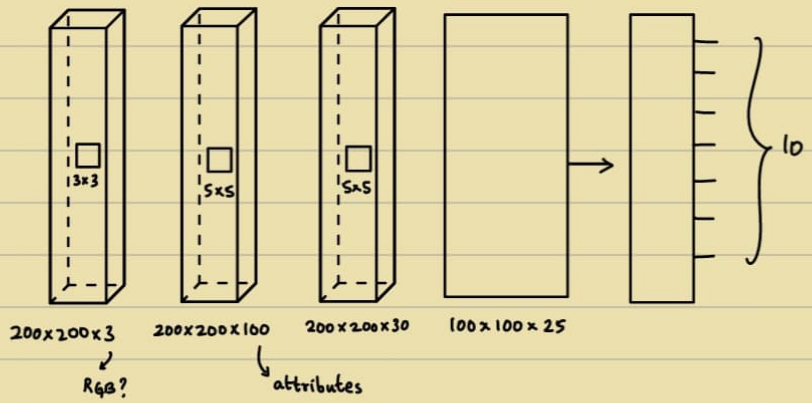
Stride

Pool



d' Kernels





Kernel size change, $3 \times 3 \longrightarrow 5 \times 5$, Output affected ?

Stride change, $1 \rightarrow 2 \Rightarrow$ Output affected

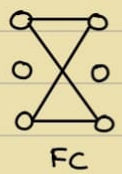
- How many params, filters, etc ?

No. of learnable params : $H_k \times W_k \times C_{in} \times C_{out}$

$$H \times W \times C_{in} \longrightarrow \underset{\text{(Kernel)}}{H_k \times W_k \times C_{out}} \longrightarrow H' \times W' \times C_{in}$$

Image Taken in camera $\longrightarrow$ reduced to $200 \times 200 \times 3 = 120K$

Locally connected layer is better



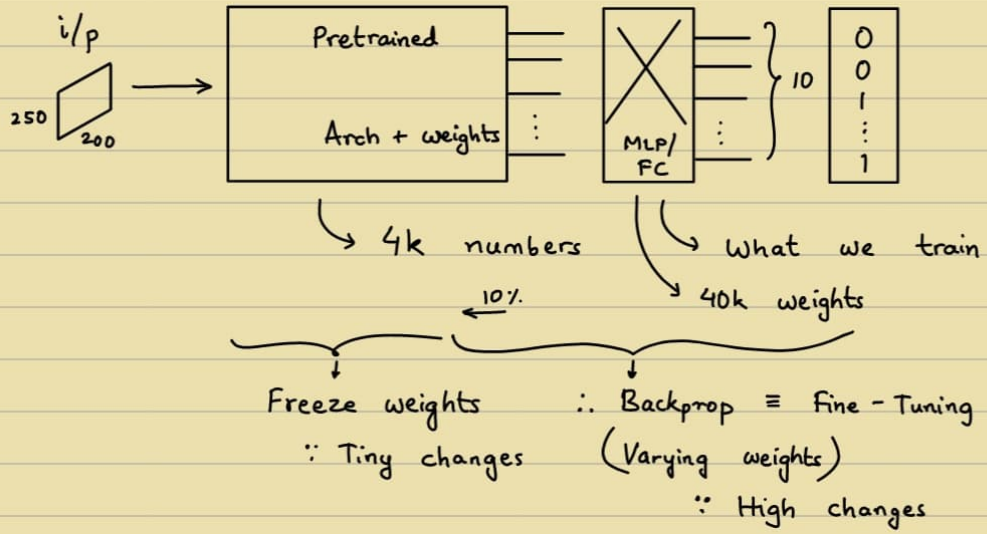
$120K \times 120K = 14\text{4 B}$

Weight Sharing ← Reusable weights

$100 \times 3 \times 3 \times 3 = 2700$ learnable weights

$\therefore$ 14.4 M $\longrightarrow$ 2700 weights

Borrow Arch. and weights (trained)



① Pretrained + ② Finetune

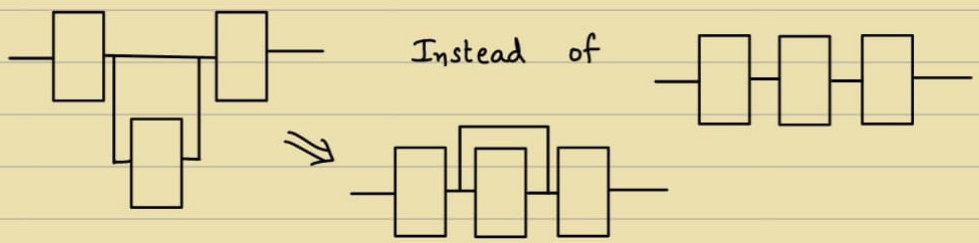
Pretrained → Reusable

Finetune → Task Specific

Task domain vs. Pretrained domain

Early / Lowest layers learn highly common patterns
& Also converge quicker
e.g. Shapes, Colours

Training data, Deep network performs bad
Vanishing Gradient Problem
Sol. : Residual / Skip connections



Extra layers ≡ Identity layers

$$\frac{\partial h}{\partial x} = \frac{\partial h}{\partial y} \cdot \frac{\partial y}{\partial x}$$

$$= \frac{\partial h}{\partial y} \cdot (1 + F'(x))$$

RESNET

DENSENET

Q) Long doc. seq. / long vids.

↳ How long for Vanishing gradients ? with RNN

long range dependency (Parallelizable)

Do LSTM solve the problem ?

Any cases where RNN still used ? ✓

Q) CNN trained on ImageNet

↳ different distribution (slight) Natural (Camera, Adversarial, Style)

Robustness ? - Medical

Q) Shape biased CNN w.r.t OOD Robustness

Shape biased / colour biased - why ? (Spurious shortcuts)

Easy to learn, why ?

Any architectural tricks to reduce (Overfit to bg rather than obj.)

Q) RNN - training slow, GPU ↑ (inputs step by step)

(No parallelization)

∴ Transformers

↳ Memory & latency

Parallelizable

[Video]

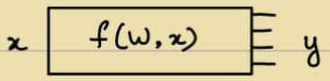
→ Learning structure ? Frames → Shots → Scene

Word → Sequence → Section

Combination of CNN and RNN

9/4

→ More on CNNs :



$\hat{y} = f(w, x)$

$\min_w |y \sim f(w,x)|$ ← Backpropagation problem

i.e. Find w S.T. o/p is class 2

$x^* = \max_x P(\text{class} = 3)$

Find x S.T o/p is class 2 ?

i.e. max score for class

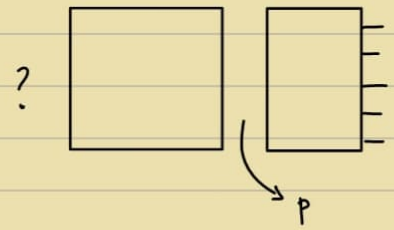
init x – random

$\frac{\partial h}{\partial x}$ & $x^{t+1} = x^t - \eta \frac{\partial h}{\partial x}$

Backprop. : $w^* = \text{opt}_w \; f(w,x)$

$x^* = \text{opt}_x \; f(w,x)$ ↘ What is model learning ?

What could have been input at this level to maximise the output

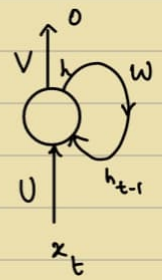


→ RNN :

Recursion

No feedback / memory , only feed-forward

∵ Vanishing Grad. Problem



$o = Vh$

$h = Ux$ [hidden state]

$h_t = f(Ux_t + Wh_{t-1})$

$o_t = \text{Softmax}(Vh_t)$

f : activation function

h_0 : Initialisation

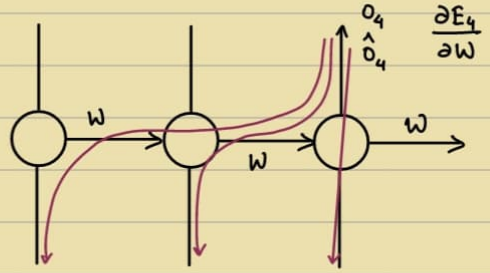
U, V, W : learn by BP

- Andrej Karpathy - Next character sequence

  e.g. hello : $\{(h,e), (e,l), (l,l), (l,o)\} \equiv \{(x,o)\}$

  Every character is encoded. (one-hot)

- RNN - Infinite Memory
- RNN : one to one
  one to many
  many to one
  many to many



Time Unwrapped version

Backpropagation

$E_4 = (O_4 - \hat{O}_4)^2$

$$\frac{\partial E_4}{\partial W} = \frac{\partial E_4}{\partial \hat{O}_4} \cdot \frac{\partial \hat{O}_4}{\partial h_4} \cdot \frac{\partial h_4}{\partial W}$$ (Not enough)

h_4 depends on h_3

h_3 depends on W

i.e. $h_4 = f(Ux_3 + Wh_3)$

$$\frac{\partial E_4}{\partial W} = \frac{\partial E_4}{\partial \hat{O}_4} \cdot \frac{\partial \hat{O}_4}{\partial h_4} \cdot \frac{\partial h_4}{\partial h_3} \cdot \frac{\partial h_3}{\partial W}$$

$$\frac{\partial E_4}{\partial W} = \frac{\partial E_4}{\partial \hat{O}_4} \cdot \frac{\partial \hat{O}_4}{\partial h_4} \cdot \frac{\partial h_4}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial W}$$

BP through time

Vanishing Gradient becomes a problem - Hard to solve

Exploding Gradient - Problem but controllable

(Clamp the explosion)

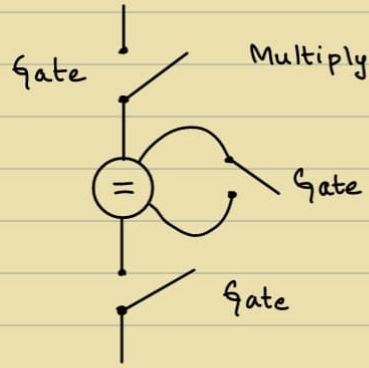
→ LSTM:

Long Short-Term Memory

Read, Write and Erase / Forget

[Input]

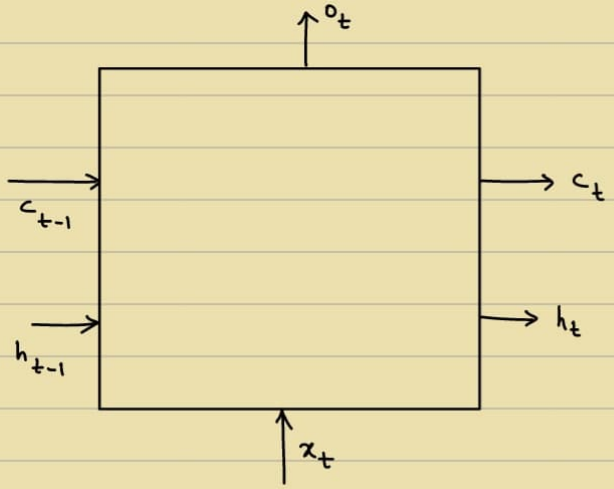
Gating Mechanism - Input Gate, Forget Gate & Output Gate



Why not {0,1}? ← [0,1] ✓

When gates open / close details comes from the data itself

Gates : Soft Controllable



x_t and h_{t-1} → inputs

Task, Data, Loss gives you the W matrix

13/6

RNNs

LSTMs

GRU

ARMA : Auto Regressive with Moving Averages

- Embedding :
  - ↳ Can exist for any entity
    - X2Vec
    - Wor2Vec Representation

w_1 — w_3 w_4

Task : Predict missing word

Weight Matrix : Rows as Vectors

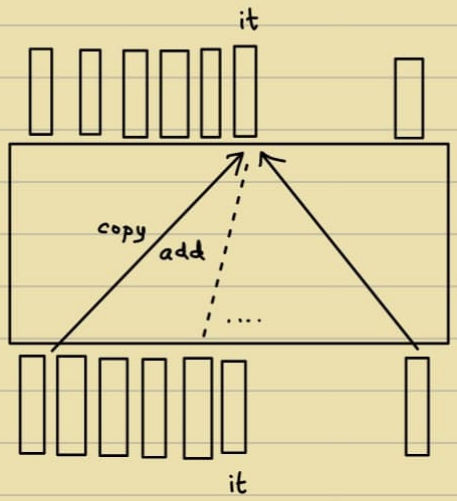
1 Hot $\xrightarrow[\text{Mult.}]{\text{Matrix}}$ Word Vec.

→ Self-Attention for Transformers :

Encoder : Understand Input

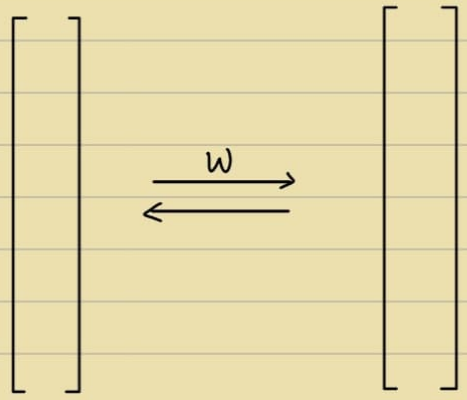
Decoder : Generate Output

Self Attention Layer : Has embeddings corresponding to all words.



W matrix assigns it to be 1 x animal + 0 x road

0 x animal + 1 x road



W matrix depends on the input

In MLP, CNN, etc, input does not matter for W.

- Tokens :

Combined to form words

$$\begin{bmatrix} a \\ b \\ \vdots \\ z \end{bmatrix} + \begin{bmatrix} ing \end{bmatrix} + \dots$$

Simple Self-Attention :

$$y_i = \sum_j w_{ij} x_j$$

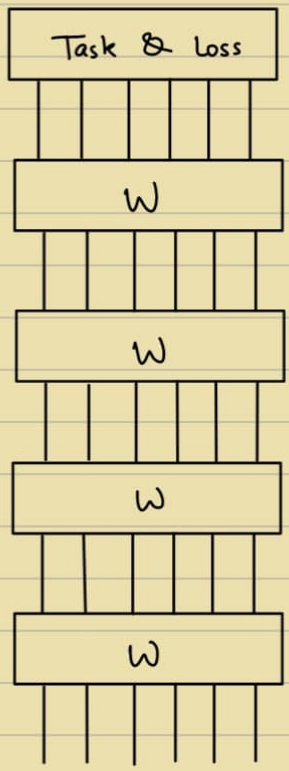
$$w'_{ij} = x_i^T x_j$$

$$w_{ij} = \frac{\exp w'_{ij}}{\sum_k \exp w'_{ik}}$$

Softmax → $\sum = 1$

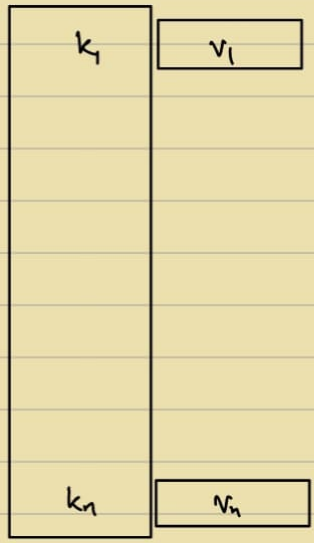
W here are not learnable.

input and output : same dimensions



$n \times d$

Key, Value & Query

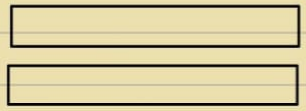


Query q

if $(q == v_i)$
return v_i
} Vanilla Approach

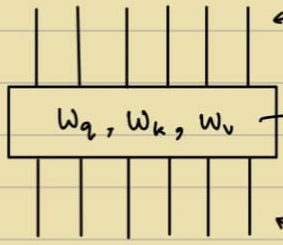
Instead return $\sum \sim q \times v_i$

e.g. 2 words



We find $q_i^T k_j$

QK^T



Responsible for generating key-value pairs

$n \times d$

↳ words or Tokens

↳ dimensions

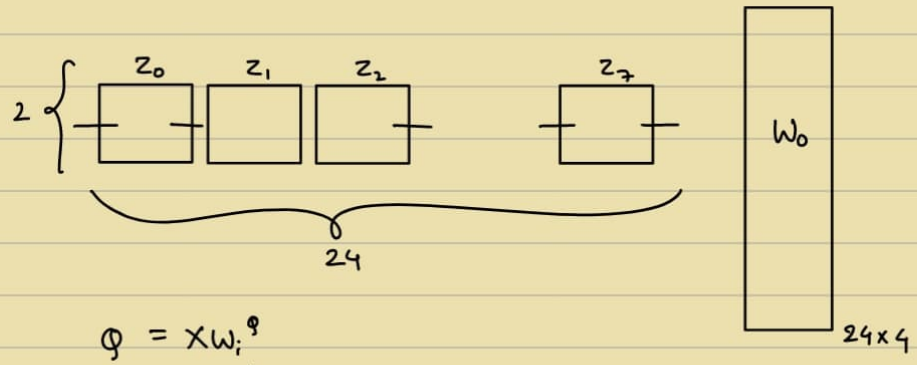
Learnable params. : d_k (= d_q), d_q, d_v

W_o converts d to d_v.

Self Attention vs. Cross Attention

Multi Head Attention

Concatenate all matrices



$Q = XW_i^Q$

$K = XW_i^K$

$V = XW_i^V$

Sets ?

Future ? ← Sequences ?

QK^T has $-\infty$ in upper half

$\Rightarrow$ 0 after softmax

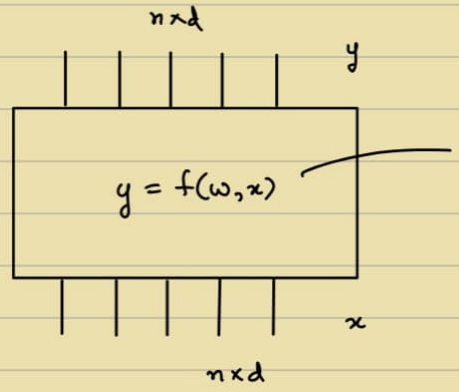
[Masked Attention]

RNN vs. 1D Conv. vs. Self-Attention

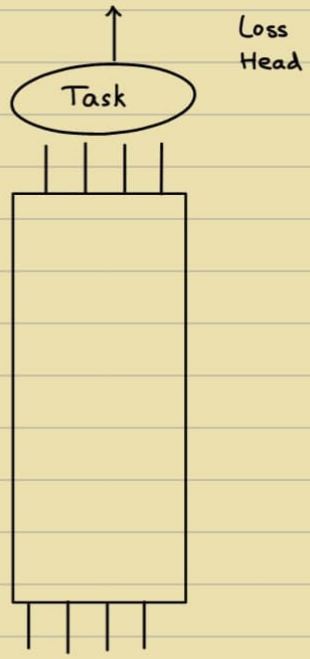
↳ Takes Adv. of RNN & Conv.

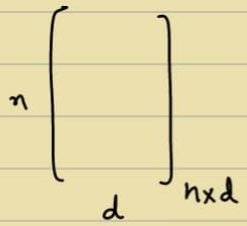
Disadv. → Memory Intensive

16/4



every y_i depends on every x_i $\Rightarrow$ y_i's are enriched version of x_i





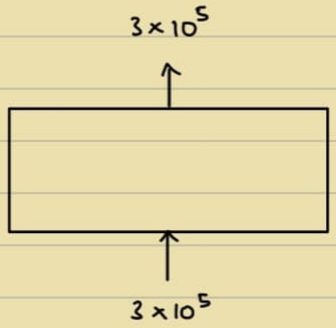
$q_i^T k_j \quad \forall i \; \forall j$

i & j → n vectors

$n \times n$

$y_j = \sum_i \alpha_{ji} v_i \;, \quad Z = AV$

Multi head,
final value : Y



$N = 1000$

$d = 300$

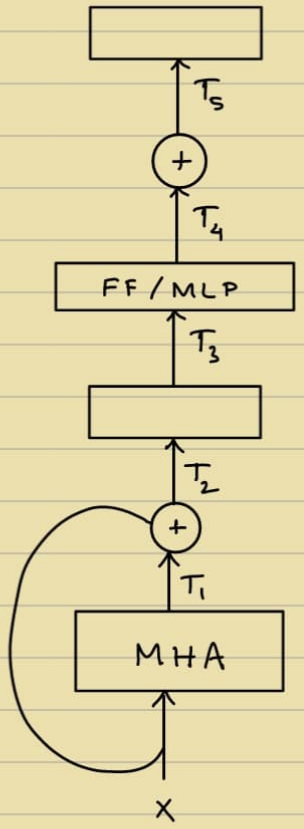
MLP : $3 \times 10^5 \times 3 \times 10^5$

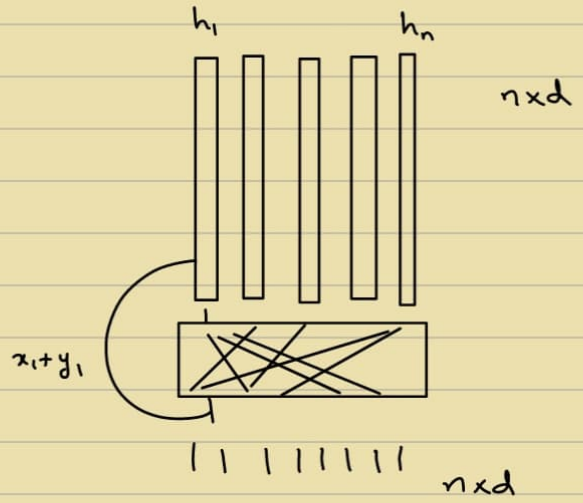
CNN : $300 \times 3 \times 3 \times 300$

RNN : U (300×100)
W (100×100)
V (100×300)
} x

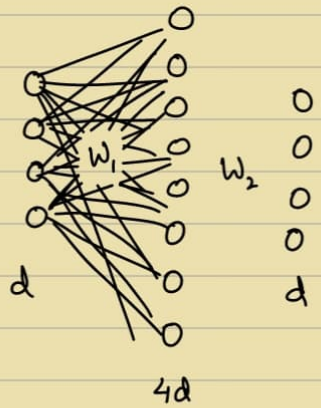
Say $h = 100$

SA : W_q (300×100)
W_k (300×100)
W_v (300×100)
W_o (100×300)





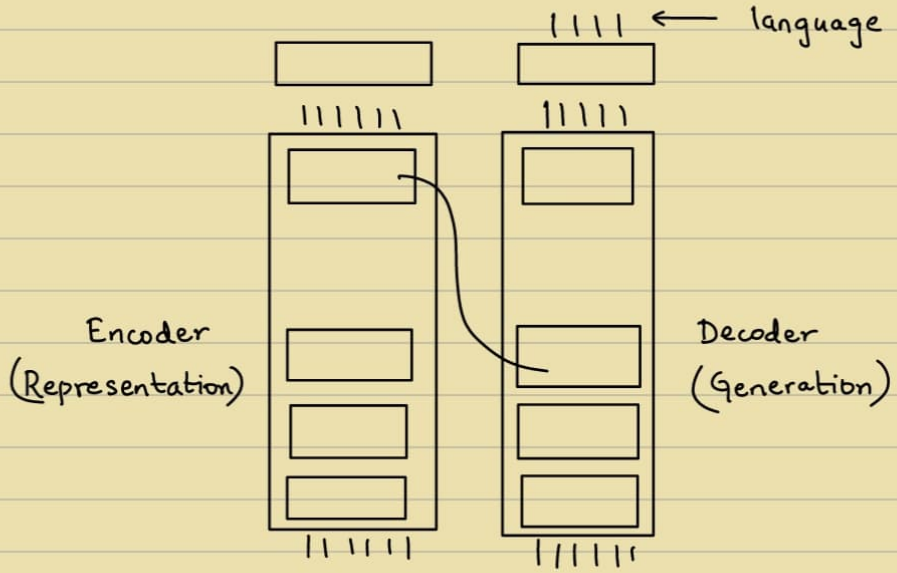
Normalise within tokens



Normalise within vector

↳ LayerNorm , Skip conn.

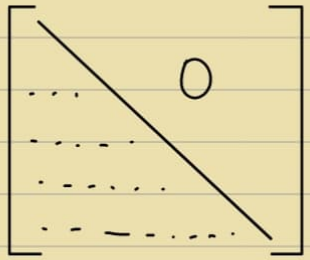
→ Transformers :



- Encoder to Decoder → Meaning of a sentence & asks to generate (Not english meaning)

Reduce Training Time : Masked Attention

A = (mixing)

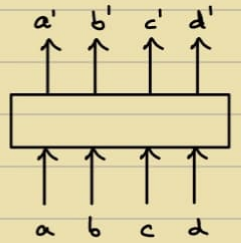


Teacher Forcing

↳ Give inputs into decoder, Output may be diff.

Keys & Values → Encoder

Queries → Decoder



Self Attention : $n \times d$

Cross Attention : $n \times d$ and $m \times d$

- Softmax ⟹ Words ?

  i.e. Language Modeling Head

  When Language is required.

ImageNet Moment

Pretraining ⟹ Fine-tuning

Sentence Completion

Vision Transformer

How they are trained — Self-Supervised Learning

20/4

Cost of Data ↑

Train without Supervision ⟹ Fine-Tune

Unlabeled data ↑

- PEFT :

$A' = A + B$

- X-ray : Self - Supervision

- $d(x,y) = \|X-Y\|$
$= [X-Y]^T A [X-Y]$

Cholesky decomposition : $A : LL^T$

$[L^TX - LY]^T [L^TX - LY]$ ← Transformation & then Euclidean dist.

Metric Learning → Similar Samples come closer
Dissimilar Samples go farther

Instead of (X, Y), We do $((X_1, X_2), 1)$
$((X_1, X_3), 0)$
↓ ↓
X Y

Siamese Arch. → Single loss function

23/4

The End !